```
import { createApp } from 'vue'
import App from './App.vue'
import router from './router'
import './style.css'
import './sender-font-size.css'
import './split-layout.css'
import { softwareTitle } from './software'
document.title = softwareTitle
createApp(App).use(router).mount('#app')
export const software = import.meta.env.VITE_SOFTWARE === 'monitor' ? 'monitor'
  : import.meta.env.VITE_SOFTWARE === 'prediction' ? 'prediction' : 'adaptive'
export const softwareTitle = {
  monitor: '工业网络传输状态实时感知与监测系统',
  adaptive: '运行监测数据自适应压缩传输控制决策与优化系统',
  prediction: '工业网络数据传输智能诊断与预测系统'
}[software]
<script setup lang="ts">
import { computed, onBeforeUnmount, onMounted, ref } from 'vue'
import TransmissionControls from './TransmissionMonitorView.vue'
import PanelTitle from '@/components/PanelTitle.vue'
import NetworkPredictionChart from
'@/components/NetworkPredictionChart.vue'
import { dashboardApi } from '@/api/dashboard'
import type { StrategyDashboard, EffectDashboard, TransmissionDashboard } from '@/types/dashboard'
const strategy = ref<StrategyDashboard | null>(null)
const effect = ref<EffectDashboard | null>(null)
const transmission = ref<TransmissionDashboard | null>(null)
const message = ref('')
const transferMetrics = computed(() => ['平均分片时延', '平均传输速率', '分片抖动'].map(label => ({ label, value: effect.value?.network?.metrics?.find(item => item.label === label)?.value || '–', note: '完成任务统计' })))
const liveMetrics = computed(() => [
  ...(transmission.value?.route.metrics || []).filter(item => item.label !== '当前链路状态'),
  ...(transmission.value?.keyMetrics || []).filter(item => ['平均有效吞吐率', '网络吞吐率', '重试次数', '完整性校验'].includes(item.label)),
  { label: '传输进度', value: transmission.value ? `${transmission.value.progress.value}%` : '—' },
  { label: '已处理帧/总帧', value: transmission.value?.progress.chunkProgress || '—' }
].map(item => ({ label: item.label, value: item.value, note: '链路与任务实时' })))
let timer: number | undefined
let pending = false
async function load() {
  if (pending) return
  pending = true
  try {
    const [nextStrategy, nextEffect] = await Promise.all([dashboardApi.strategy(), dashboardApi.effect()])
    strategy.value = nextStrategy
    effect.value = nextEffect
    message.value = ''
  } catch (error) {
    message.value = error instanceof Error ? error.message : '无法获取网络速率预测数据'
  } finally { pending = false }
}
onMounted(async () => { await load(); timer = window.setInterval(load, 1000) })
onBeforeUnmount(() => { if (timer) window.clearInterval(timer) })
```

```
</script>
<template>
  <div class="split-software prediction-software">
    <TransmissionControls controls-only @dashboard="transmission = $event" />
    <div v-if="message" class="toast-message error" role="status">{{ message }}</div>
    <section v-if="strategy" class="panel strategy-chart-panel network-panel prediction-analysis">
      <PanelTitle title="长短期记忆网络速率预测对比" icon="↘" />
      <NetworkPredictionChart :points="strategy.networkForecast.points" />
      <div class="forecast-metrics">
        <div v-for="item in strategy.networkForecast.metrics" :key="item.label"><span>{{ item.label }}</span><b>{{ item.value }}</b></div>
      </div>
    </section>
    <section v-if="effect" class="panel monitor-metrics-panel">
      <div class="section-heading"><h2>关键指标 <small>（任务与链路）</small></h2><label>刷新频率：<b>1
秒</b></label></div>
      <div class="monitor-metrics-grid prediction-kpis">
        <article v-for="item in [...transferMetrics, ...liveMetrics]" :key="item.label" class="kpi-card"><div><small>{{ item.label }}</small><strong>{{ item.value }}</strong><em>{{ item.note }}</em></div></article>
      </div>
    </section>
    <div v-if="!strategy && !message" class="empty-panel">正在加载网络速率预测数据...</div>
  </div>
</template>
package com.datacompress.control;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableScheduling;
@SpringBootApplication
@EnableScheduling
public class DataCompressControlApplication {
    public static void main(String[] args) {
        SpringApplication.run(DataCompressControlApplication.class, args);
    }
}
package com.datacompress.control.service;
import com.datacompress.control.domain.NetworkSample;
import com.datacompress.control.domain.PredictionHealth;
import com.datacompress.control.domain.RateForecast;
import com.datacompress.control.domain.TelemetryStatus;
import com.fasterxml.jackson.databind.ObjectMapper;
import jakarta.annotation.PostConstruct;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.core.io.ClassPathResource;
import org.springframework.stereotype.Service;
import java.io.InputStream;
import java.time.Instant;
import java.util.*;
@Service
public class CompactLstmRatePredictor {
    private static final Logger log = LoggerFactory.getLogger(CompactLstmRatePredictor.class);
```

```
    private final ObjectMapper mapper;
    private final PredictionFeedbackService feedback;
    private final NetworkTelemetryService network;
    private final int configuredHorizon;
    private final int historySamples;
    private final double minimumRateKbps;
    private Model model;
    public CompactLstmRatePredictor(
            ObjectMapper mapper,
            PredictionFeedbackService feedback,
            NetworkTelemetryService network,
            @Value("${app.predictor.horizon:12}") int configuredHorizon,
            @Value("${app.predictor.history-samples:7200}") int historySamples,
            @Value("${app.network.minimum-fallback-rate-kbps:64}") double
minimumRateKbps) {
        this.mapper = mapper;
        this.feedback = feedback;
        this.network = network;
        this.configuredHorizon = Math.max(1, configuredHorizon);
        this.historySamples = Math.max(120, historySamples);
        this.minimumRateKbps = Math.max(1.0, minimumRateKbps);
    }
    @PostConstruct
    public void load() {
        try (InputStream in = new ClassPathResource("model/rate-lstm.json").getInputStream()) {
            model = mapper.readValue(in, Model.class);
            validateModel(model);
            log.info("Loaded compact LSTM model: sequenceLength={}, horizon={},
hiddenSize={}",
                    model.sequenceLength, model.horizon, model.hiddenSize);
        } catch (Exception error) {
            model = null;
            log.error("LSTM model unavailable; robust fallback will remain active", error);
        }
    }
    public RateForecast predictCurrent() {
        NetworkSample current = network.current();
        return predict(network.rateHistory(historySamples),
                current.packetLossPercent(), current.jitterMillis());
    }
    public RateForecast predict(List<Double> history, double lossPercent, double jitterMillis) {
        long started = System.nanoTime();
        List<Double> clean = history == null ? List.of() : history.stream()
                .filter(v -> v != null && Double.isFinite(v) && v > 0)
                .toList();
        Baseline baseline = baseline(clean);
        TelemetryStatus telemetry = network.status();
        PredictionHealth health = feedback.health();
        boolean telemetryDegraded = !telemetry.fresh();
        boolean modelDegraded = !health.modelAllowed();
        List<Double> predicted;
        String modelName;
```

```
        String degradationReason = "NONE";
        if (telemetryDegraded) {
            predicted = robustFallback(clean, configuredHorizon, baseline);
            modelName = "ROBUST_P20_TELEMETRY_FALLBACK";
            degradationReason = "telemetry " + telemetry.source()
                    + ", ageMs=" + telemetry.ageMillis();
        } else if (model == null || clean.size() < Math.max(8, model.sequenceLength / 2)) {
            predicted = fallback(clean, configuredHorizon);
            modelName = "FAST_HOLT_WARMUP";
            degradationReason = model == null ? "model unavailable" : "insufficient history";
        } else if (modelDegraded) {
            predicted = robustFallback(clean, configuredHorizon, baseline);
            modelName = "ROBUST_P20_MODEL_CIRCUIT_BREAKER";
            degradationReason = health.reason();
        } else {
            predicted = stabilize(lstm(clean), baseline);
            modelName = "COMPACT_LSTM_MULTISCALE_GUARDED";
        }
        double average = predicted.stream().mapToDouble(Double::doubleValue)
                .average().orElse(Math.max(minimumRateKbps,
baseline.shortAverage));
        double freshnessFactor = freshnessFactor(telemetry);
        double conservativeCeiling = conservativeCeiling(baseline,
                telemetryDegraded || modelDegraded);
        double safe = feedback.safe(average, lossPercent, jitterMillis,
                freshnessFactor, conservativeCeiling);
        long elapsed = (System.nanoTime() - started) / 1_000_000L;
        PredictionHealth finalHealth = feedback.health();
        boolean degraded = telemetryDegraded || modelDegraded
                || modelName.contains("FALLBACK") ||
modelName.contains("WARMUP");
        return new RateForecast(Instant.now(), predicted, average, safe,
                finalHealth.rollingMaeKbps(), finalHealth.rollingMapePercent(),
                finalHealth.correctionFactor(), modelName, elapsed,
                telemetry.fresh(), telemetry.ageMillis(), degraded, degradationReason,
                baseline.p20, baseline.shortAverage, baseline.mediumAverage,
                finalHealth.unsafePredictionRatioPercent(), finalHealth.rollingSamples());
    }
    public PredictionHealth health() {
        return feedback.health();
    }
    private List<Double> lstm(List<Double> history) {
        int seq = model.sequenceLength;
        List<Double> tail = new ArrayList<>();
        if (history.size() >= seq) {
            tail.addAll(history.subList(history.size() - seq, history.size()));
        } else {
            double first = history.isEmpty() ? minimumRateKbps : history.get(0);
            for (int i = history.size(); i < seq; i++) tail.add(first);
            tail.addAll(history);
        }
        double mean = tail.stream().mapToDouble(Double::doubleValue).average()
```

```
                .orElse(minimumRateKbps);
        double variance = 0;
        for (double value : tail) variance += (value - mean) * (value - mean);
        double std = Math.max(20.0, Math.sqrt(variance / tail.size()));
        double[] h = new double[model.hiddenSize];
        double[] c = new double[model.hiddenSize];
        for (double value : tail) {
            double x = (value - mean) / std;
            double[] gates = new double[4 * model.hiddenSize];
            for (int row = 0; row < gates.length; row++) {
                double sum = model.bias[row] + model.weightIH[row][0] * x;
                for (int j = 0; j < model.hiddenSize; j++) {
                    sum += model.weightHH[row][j] * h[j];
                }
                gates[row] = sum;
            }
            double[] nextH = new double[model.hiddenSize];
            double[] nextC = new double[model.hiddenSize];
            for (int j = 0; j < model.hiddenSize; j++) {
                double inputGate = sigmoid(gates[j]);
                double forgetGate = sigmoid(gates[model.hiddenSize + j]);
                double cellGate = Math.tanh(gates[2 * model.hiddenSize + j]);
                double outputGate = sigmoid(gates[3 * model.hiddenSize + j]);
                nextC[j] = forgetGate * c[j] + inputGate * cellGate;
                nextH[j] = outputGate * Math.tanh(nextC[j]);
            }
            h = nextH;
            c = nextC;
        }
        int horizon = Math.min(configuredHorizon, model.horizon);
        double historicalMin = Collections.min(tail);
        double historicalMax = Collections.max(tail);
        double lower = Math.max(minimumRateKbps, historicalMin * 0.45);
        double upper = Math.max(lower, historicalMax * 1.55);
        List<Double> out = new ArrayList<>(horizon);
        for (int i = 0; i < horizon; i++) {
            double y = model.outputBias[i];
            for (int j = 0; j < model.hiddenSize; j++) {
                y += model.outputWeight[i][j] * h[j];
            }
            double rate = mean + y * std;
            out.add(clamp(rate, lower, upper));
        }
        return out;
    }
    private List<Double> stabilize(List<Double> raw, Baseline baseline) {
        List<Double> out = new ArrayList<>(raw.size());
        for (int i = 0; i < raw.size(); i++) {
            double horizonWeight = Math.exp(-i / 5.0);
            double anchor = baseline.mediumAverage
```

```
                        + (baseline.shortAverage - baseline.mediumAverage) * horizonWeight;
                double blended = raw.get(i) * 0.72 + anchor * 0.23 + baseline.p20 * 0.05;
                double lower = Math.max(minimumRateKbps, baseline.p10 * 0.75);
                double upper = Math.max(lower, baseline.p90 * 1.30);
                out.add(clamp(blended, lower, upper));
            }
            return out;
        }
        private List<Double> robustFallback(List<Double> values, int horizon, Baseline baseline) {
            if (values.isEmpty()) return Collections.nCopies(horizon, minimumRateKbps);
            double level = Math.min(baseline.shortAverage, baseline.mediumAverage);
            double target = Math.max(baseline.p20, Math.min(level, baseline.p35));
            double trend = clamp((baseline.shortAverage - baseline.mediumAverage) * 0.20,
                    -target * 0.10, target * 0.05);
            List<Double> out = new ArrayList<>(horizon);
            for (int i = 1; i <= horizon; i++) {
                double value = target + trend * i * Math.exp(-i / 3.0);
                out.add(Math.max(minimumRateKbps, value));
            }
            return out;
        }
        private List<Double> fallback(List<Double> values, int horizon) {
            if (values.isEmpty()) return Collections.nCopies(horizon, minimumRateKbps);
            int count = Math.min(24, values.size());
            List<Double> tail = values.subList(values.size() - count, values.size());
            double level = tail.get(0);
            double trend = 0;
            for (int i = 1; i < tail.size(); i++) {
                double previousLevel = level;
                level = 0.35 * tail.get(i) + 0.65 * (level + trend);
                trend = 0.15 * (level - previousLevel) + 0.85 * trend;
            }
            List<Double> out = new ArrayList<>();
            for (int i = 1; i <= horizon; i++) {
                out.add(Math.max(minimumRateKbps,
                        level + trend * i * Math.pow(0.96, i)));
            }
            return out;
        }
        private Baseline baseline(List<Double> values) {
            if (values.isEmpty()) {
                return new Baseline(minimumRateKbps, minimumRateKbps,
minimumRateKbps,
                        minimumRateKbps, minimumRateKbps,
minimumRateKbps);
            }
            List<Double> shortWindow = tail(values, 24);
            List<Double> mediumWindow = tail(values, 120);
            return new Baseline(
                    percentile(mediumWindow, 0.10),
                    percentile(mediumWindow, 0.20),
                    percentile(mediumWindow, 0.35),
```

```
                percentile(mediumWindow, 0.90),
                average(shortWindow),
                average(mediumWindow));
    }
    private double conservativeCeiling(Baseline baseline, boolean degraded) {
        double healthy = Math.max(minimumRateKbps,
                Math.min(baseline.p90, baseline.p35 * 1.10));
        if (!degraded) return healthy;
        return Math.max(minimumRateKbps, baseline.p20 * 0.90);
    }
    private double freshnessFactor(TelemetryStatus status) {
        if (status.fresh()) return 1.0;
        if (status.ageMillis() == Long.MAX_VALUE) return 0.35;
        double factor = 0.75 - Math.min(0.45, status.ageMillis() / 60_000.0 * 0.45);
        return clamp(factor, 0.30, 0.75);
    }
    private List<Double> tail(List<Double> values, int count) {
        int from = Math.max(0, values.size() - count);
        return values.subList(from, values.size());
    }
    private double average(List<Double> values) {
        return values.stream().mapToDouble(Double::doubleValue).average()
                .orElse(minimumRateKbps);
    }
    private double percentile(List<Double> values, double quantile) {
        if (values.isEmpty()) return minimumRateKbps;
        List<Double> sorted = new ArrayList<>(values);
        sorted.sort(Double::compareTo);
        int index = (int)Math.floor(clamp(quantile, 0, 1) * (sorted.size() - 1));
        return sorted.get(index);
    }
    private void validateModel(Model value) {
        if (value.sequenceLength <= 0 || value.horizon <= 0 || value.hiddenSize <= 0
                || value.weightIH == null || value.weightHH == null || value.bias == null
                || value.outputWeight == null || value.outputBias == null) {
            throw new IllegalArgumentException("Incomplete LSTM model");
        }
        if (value.bias.length != value.hiddenSize * 4
                || value.outputBias.length < Math.min(configuredHorizon, value.horizon)) {
            throw new IllegalArgumentException("Invalid LSTM dimensions");
        }
    }
    private static double sigmoid(double x) {
        if (x >= 0) {
            double z = Math.exp(-x);
            return 1.0 / (1.0 + z);
        }
        double z = Math.exp(x);
        return z / (1.0 + z);
    }
```

```
    private static double clamp(double value, double min, double max) {
        return Math.max(min, Math.min(max, value));
    }
    private record Baseline(double p10, double p20, double p35, double p90,
                            double shortAverage, double mediumAverage) {}
    public static class Model {
        public String modelType;
        public int version;
        public int sequenceLength;
        public int horizon;
        public int inputSize;
        public int hiddenSize;
        public double trainingLoss;
        public double[][] weightIH;
        public double[][] weightHH;
        public double[] bias;
        public double[][] outputWeight;
        public double[] outputBias;
    }
}
package com.datacompress.control.service;
import com.datacompress.control.domain.PredictionHealth;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.ArrayDeque;
import java.util.Deque;
@Service
public class PredictionFeedbackService {
    private final double alpha;
    private final double minimum;
    private final double maximum;
    private final int windowSize;
    private final int minimumHealthSamples;
    private final double maximumMapePercent;
    private final double maximumUnsafeRatio;
    private final long cooldownMillis;
    private final double maximumSafeStepUp;
    private final Deque<ErrorPoint> rolling = new ArrayDeque<>();
    private double correctionFactor = 0.90;
    private double mae;
    private double mape;
    private double unsafeRatio;
    private long degradedUntil;
    private String degradationReason = "HEALTHY";
    private double lastSafeRate;
    public PredictionFeedbackService(
            @Value("${app.predictor.feedback-alpha:0.18}") double alpha,
            @Value("${app.predictor.minimum-safe-factor:0.55}") double
minimum,
            @Value("${app.predictor.maximum-safe-factor:1.05}") double
maximum,
```

```
            @Value("${app.predictor.health-window-samples:240}") int windowSize,
            @Value("${app.predictor.health-minimum-samples:30}") int
minimumHealthSamples,
            @Value("${app.predictor.health-maximum-mape-percent:35}") double
maximumMapePercent,
            @Value("${app.predictor.health-maximum-unsafe-ratio:0.35}") double
maximumUnsafeRatio,
            @Value("${app.predictor.health-cooldown-ms:60000}") long cooldownMillis,
            @Value("${app.predictor.maximum-safe-step-up:1.15}") double
maximumSafeStepUp) {
        this.alpha = clamp(alpha, 0.01, 1.0);
        this.minimum = Math.max(0.05, minimum);
        this.maximum = Math.max(this.minimum, maximum);
        this.windowSize = Math.max(20, windowSize);
        this.minimumHealthSamples = Math.max(10, Math.min(this.windowSize,
minimumHealthSamples));
        this.maximumMapePercent = Math.max(1.0, maximumMapePercent);
        this.maximumUnsafeRatio = clamp(maximumUnsafeRatio, 0.05, 1.0);
        this.cooldownMillis = Math.max(5_000L, cooldownMillis);
        this.maximumSafeStepUp = clamp(maximumSafeStepUp, 1.0, 2.0);
    }
    public synchronized void update(double predicted, double actual) {
        if (!Double.isFinite(predicted) || predicted <= 0
                || !Double.isFinite(actual) || actual <= 0) return;
        double ratio = clamp(actual / predicted, minimum, maximum);
        correctionFactor = alpha * ratio + (1.0 - alpha) * correctionFactor;
        double error = Math.abs(actual - predicted);
        double percent = error / actual * 100.0;
        boolean unsafe = predicted > actual * 1.15;
        rolling.addLast(new ErrorPoint(error, percent, unsafe));
        while (rolling.size() > windowSize) rolling.removeFirst();
        recalculate();
        evaluateHealth();
    }
    public synchronized double safe(double predicted, double lossPercent, double jitterMillis) {
        return safe(predicted, lossPercent, jitterMillis, 1.0, Double.POSITIVE_INFINITY);
    }
    public synchronized double safe(double predicted, double lossPercent, double jitterMillis,
                                    double freshnessFactor, double
conservativeCeiling) {
        double penalty = clamp(1.0 - Math.max(0, lossPercent) / 100.0
                        - Math.max(0, jitterMillis) / 1000.0,
                0.55, 1.0);
        double target = Math.max(1.0, predicted * correctionFactor * penalty
                * clamp(freshnessFactor, 0.20, 1.0));
        if (Double.isFinite(conservativeCeiling) && conservativeCeiling > 0) {
            target = Math.min(target, conservativeCeiling);
        }
        if (lastSafeRate > 0) {
            target = Math.min(target, lastSafeRate * maximumSafeStepUp);
        }
        lastSafeRate = target;
        return target;
    }
    public synchronized PredictionHealth health() {
        long remaining = Math.max(0L, degradedUntil - System.currentTimeMillis());
```

```
            boolean degraded = remaining > 0;
            return new PredictionHealth(Instant.now(), !degraded, degraded,
                    degraded ? degradationReason : "HEALTHY",
                    rolling.size(), mae, mape, unsafeRatio * 100.0,
                    correctionFactor, remaining);
        }
        public synchronized double factor() { return correctionFactor; }
        public synchronized double mae() { return mae; }
        public synchronized double mape() { return mape; }
        public synchronized double unsafeRatioPercent() { return unsafeRatio * 100.0; }
        public synchronized int sampleCount() { return rolling.size(); }
        private void recalculate() {
            if (rolling.isEmpty()) {
                mae = mape = unsafeRatio = 0;
                return;
            }
            double errorSum = 0;
            double percentSum = 0;
            int unsafe = 0;
            for (ErrorPoint point : rolling) {
                errorSum += point.absoluteError();
                percentSum += point.percentError();
                if (point.unsafe()) unsafe++;
            }
            mae = errorSum / rolling.size();
            mape = percentSum / rolling.size();
            unsafeRatio = unsafe / (double) rolling.size();
        }
        private void evaluateHealth() {
            if (rolling.size() < minimumHealthSamples) return;
            String reason = null;
            if (mape > maximumMapePercent) {
                reason = String.format("MAPE %.1f%% > %.1f%%", mape,
maximumMapePercent);
            } else if (unsafeRatio > maximumUnsafeRatio) {
                reason = String.format("unsafe ratio %.1f%% > %.1f%%",
                        unsafeRatio * 100.0, maximumUnsafeRatio * 100.0);
            }
            if (reason != null) {
                degradedUntil = Math.max(degradedUntil, System.currentTimeMillis() +
cooldownMillis);
                degradationReason = reason;
                correctionFactor = Math.min(correctionFactor, 0.75);
            }
        }
        private static double clamp(double value, double min, double max) {
            return Math.max(min, Math.min(max, value));
        }
        private record ErrorPoint(double absoluteError, double percentError, boolean unsafe) {}
}
<template>
  <RouterView />
```

```
</template>
<script setup lang="ts">
import * as echarts from 'echarts'
import { onBeforeUnmount, onMounted, ref, watch } from 'vue'
interface ChartSeries {
  name: string
  data: number[]
  color: string
  dashed?: boolean
  area?: boolean
}
const props = withDefaults(defineProps<{
  labels: string[]
  series: ChartSeries[]
  yName?: string
  yMax?: number
  yInterval?: number
  height?: number
  markerIndexes?: { index: number; label: string }[]
}>(), {
  yName: '',
  yMax: undefined,
  yInterval: undefined,
  height: 260,
  markerIndexes: () => []
})
const chartEl = ref<HTMLDivElement>()
let chart: echarts.ECharts | undefined
let observer: ResizeObserver | undefined
function render() {
  if (!chartEl.value) return
  const fontSizeOffset = chartEl.value.closest('.app-shell') ? 4 : 0
  chart ||= echarts.init(chartEl.value)
  chart.setOption({
    animationDuration: 420,
    grid: { left: 12, right: 18, top: 46, bottom: 20, containLabel: true },
    tooltip: {
      trigger: 'axis',
      backgroundColor: 'rgba(7, 32, 91, .92)',
      borderWidth: 0,
      textStyle: { color: '#fff', fontSize: 14 + fontSizeOffset }
    },
    legend: props.series.length > 1 ? {
      top: 2,
      right: 2,
      itemWidth: 18,
      itemHeight: 3,
      textStyle: { color: '#536281', fontSize: 13 + fontSizeOffset }
    } : undefined,
    xAxis: {
```

```
        type: 'category',
        boundaryGap: false,
        data: props.labels,
        axisLine: { lineStyle: { color: '#dfe7f7' } },
        axisTick: { show: false },
        axisLabel: { color: '#4d5d84', fontSize: 13 + fontSizeOffset, margin: 12 }
      },
      yAxis: {
        type: 'value',
        min: 0,
        max: props.yMax,
        interval: props.yInterval,
        axisLabel: { color: '#4d5d84', fontSize: 13 + fontSizeOffset, margin: 10 },
        splitLine: { lineStyle: { color: '#e7eefb', type: 'dashed' } },
        axisLine: { show: false },
        axisTick: { show: false }
      },
      series: props.series.map((item, index) => ({
        name: item.name,
        type: 'line',
        smooth: true,
        data: item.data,
        symbol: index === 0 ? 'circle' : 'none',
        symbolSize: 6,
        lineStyle: { width: 2.2, color: item.color, type: item.dashed ? 'dashed' : 'solid' },
        itemStyle: { color: item.color },
        areaStyle: item.area ? { color: new echarts.graphic.LinearGradient(0, 0, 0, 1, [{ offset: 0, color: `${item.color}42` }, { offset: 1, color: `${item.color}03` }]) } : undefined,
        markLine: index === 0 && props.markerIndexes.length ? {
          symbol: 'none',
          label: { show: true, color: '#1769ff', fontSize: 12 + fontSizeOffset, fontWeight: 700, backgroundColor: '#edf4ff', borderColor: '#8ab7ff', borderWidth: 1, borderRadius: 4, padding: [3, 5] },
          lineStyle: { color: '#8ab7ff', type: 'dashed', width: 1.3 },
          data: props.markerIndexes.map((m) => ({ xAxis: props.labels[m.index], label: { formatter: m.label } }))
        } : undefined
      }))
    }, true)
}
onMounted(() => {
  render()
  observer = new ResizeObserver(() => chart?.resize())
  if (chartEl.value) observer.observe(chartEl.value)
})
watch(() => [props.labels, props.series, props.yMax, props.yInterval], render, { deep: true })
onBeforeUnmount(() => {
  observer?.disconnect()
  chart?.dispose()
})
</script>
<template>
  <div class="chart line-chart-shell" :style="{ height: `${height}px` }">
    <span v-if="yName" class="chart-y-title">{{ yName }}</span>
```

```
      <div ref="chartEl" class="line-chart-canvas"></div>
    </div>
</template>
<script setup lang="ts">
import { computed, nextTick, onBeforeUnmount, onMounted, ref, watch } from 'vue'
export interface NetworkPredictionPoint {
    elapsedMillis: number
    actualRateKbps: number
    predictedRateKbps: number
    safeRateKbps?: number
    predictionErrorPercent?: number
    time?: string
}
const props = defineProps<{
    points: NetworkPredictionPoint[]
}>()
const canvas = ref<HTMLCanvasElement | null>(null)
const chartWrap = ref<HTMLDivElement | null>(null)
const windowMs = ref(0)
const drawnPointCount = ref(0)
const tooltipVisible = ref(false)
const tooltipLeft = ref(0)
const tooltipTop = ref(0)
const tooltipPoint = ref<NetworkPredictionPoint | null>(null)
let resizeObserver: ResizeObserver | undefined
let frame: { width: number; height: number; left: number; right: number; top: number; bottom: number; minX: number; maxX: number } | null = null
const cleanPoints = computed(() => (props.points || [])
    .filter((point) => Number.isFinite(point.elapsedMillis))
    .sort((a, b) => a.elapsedMillis - b.elapsedMillis))
const filteredPoints = computed(() => {
    const points = cleanPoints.value
    if (!points.length || windowMs.value <= 0) return points
    const latest = points[points.length - 1].elapsedMillis
    const start = Math.max(0, latest - windowMs.value)
    return points.filter((point) => point.elapsedMillis >= start)
})
const pointSummary = computed(() => {
    const count = filteredPoints.value.length
    if (!count) return '暂无数据'
    const mode = count > 20 ? '连续曲线（隐藏采样点）' : '曲线和采样点'
    const sampled = count > drawnPointCount.value ? ` · 绘图降采样至 ${drawnPointCount.value} 点` : ''
    return `当前范围 ${count} 点 · ${mode}${sampled}`
})
const tooltipStyle = computed(() => ({ left: `${tooltipLeft.value}px`, top: `${tooltipTop.value}px` }))
function lttb(data: NetworkPredictionPoint[], threshold: number): NetworkPredictionPoint[] {
    if (data.length <= threshold || threshold < 3) return data.slice()
    const sampled = [data[0]]
    const every = (data.length - 2) / (threshold - 2)
    let anchorIndex = 0
    for (let index = 0; index < threshold - 2; index += 1) {
```

```
    const averageStart = Math.floor((index + 1) * every) + 1
    const averageEnd = Math.min(Math.floor((index + 2) * every) + 1, data.length)
    let averageX = 0
    let averageY = 0
    let averageCount = 0
    for (let cursor = averageStart; cursor < averageEnd; cursor += 1) {
      averageX += data[cursor].elapsedMillis
      averageY += data[cursor].actualRateKbps
      averageCount += 1
    }
    if (!averageCount) {
      averageX = data[data.length - 1].elapsedMillis
      averageY = data[data.length - 1].actualRateKbps
      averageCount = 1
    }
    averageX /= averageCount
    averageY /= averageCount
    const rangeStart = Math.floor(index * every) + 1
    const rangeEnd = Math.min(Math.floor((index + 1) * every) + 1, data.length - 1)
    const anchor = data[anchorIndex]
    let maximumArea = -1
    let nextAnchor = rangeStart
    for (let cursor = rangeStart; cursor < rangeEnd; cursor += 1) {
      const candidate = data[cursor]
      const area = Math.abs(
        (anchor.elapsedMillis - averageX) * (candidate.actualRateKbps - anchor.actualRateKbps)
        - (anchor.elapsedMillis - candidate.elapsedMillis) * (averageY - anchor.actualRateKbps)
      )
      if (area > maximumArea) {
        maximumArea = area
        nextAnchor = cursor
      }
    }
    sampled.push(data[nextAnchor])
    anchorIndex = nextAnchor
  }
  sampled.push(data[data.length - 1])
  return sampled
}
function formatRate(value: number) {
  return Number.isFinite(value) ? `${value.toFixed(2)} KB/s` : '—'
}
function formatElapsed(milliseconds: number) {
  const total = Math.max(0, Math.round(milliseconds / 1000))
  const hours = Math.floor(total / 3600)
  const minutes = Math.floor((total % 3600) / 60)
  const seconds = total % 60
  return hours > 0
    ? `${hours}:${String(minutes).padStart(2, '0')}:${String(seconds).padStart(2, '0')}`
    : `${minutes}:${String(seconds).padStart(2, '0')}`
```

```
}
function tickLabel(value: number) {
  if (Math.abs(value) >= 1000) return value.toFixed(0)
  if (Math.abs(value) >= 100) return value.toFixed(1)
  return value.toFixed(2)
}
function draw() {
  const target = canvas.value
  const parent = chartWrap.value
  if (!target || !parent) return
  const rect = parent.getBoundingClientRect()
  const width = Math.max(320, Math.round(rect.width))
  const height = Math.max(250, Math.round(rect.height || 300))
  const ratio = window.devicePixelRatio || 1
  target.width = Math.round(width * ratio)
  target.height = Math.round(height * ratio)
  target.style.width = `${width}px`
  target.style.height = `${height}px`
  const context = target.getContext('2d')
  if (!context) return
  context.setTransform(ratio, 0, 0, ratio, 0, 0)
  context.clearRect(0, 0, width, height)
  const raw = filteredPoints.value
  if (!raw.length) {
    frame = null
    drawnPointCount.value = 0
    context.fillStyle = '#7b89a7'
    context.font = '16px "Microsoft YaHei", Arial'
    context.textAlign = 'center'
    context.fillText('暂无预测数据', width / 2, height / 2)
    return
  }
  const maximumPoints = Math.max(120, Math.min(800, Math.floor(width / 2)))
  const points = lttb(raw, maximumPoints)
  drawnPointCount.value = points.length
  const allY = points.flatMap((point) => [point.actualRateKbps, point.predictedRateKbps]).filter(Number.isFinite)
  let minX = points[0].elapsedMillis
  let maxX = points[points.length - 1].elapsedMillis
  let minY = Math.min(...allY)
  let maxY = Math.max(...allY)
  if (maxX === minX) { minX -= 500; maxX += 500 }
  if (maxY === minY) {
    const padding = Math.max(1, Math.abs(maxY) * 0.05)
    minY = Math.max(0, minY - padding)
    maxY += padding
  } else {
    const padding = Math.max(1, (maxY - minY) * 0.12)
    minY = Math.max(0, minY - padding)
    maxY += padding
  }
```

```
const margin = { left: 82, right: 24, top: 30, bottom: 58 }
const chartWidth = width - margin.left - margin.right
const chartHeight = height - margin.top - margin.bottom
const x = (value: number) => margin.left + ((value - minX) / (maxX - minX)) * chartWidth
const y = (value: number) => margin.top + (1 - (value - minY) / Math.max(1e-9, maxY - minY)) * chartHeight
frame = { width, height, left: margin.left, right: width - margin.right, top: margin.top, bottom: height - margin.bottom, minX, maxX }
context.font = '13px "Microsoft YaHei", Arial'
context.strokeStyle = '#e3eaf6'
context.fillStyle = '#71809f'
context.lineWidth = 1
for (let index = 0; index < 6; index += 1) {
  const value = minY + ((maxY - minY) * index) / 5
  const position = y(value)
  context.beginPath()
  context.moveTo(margin.left, position)
  context.lineTo(width - margin.right, position)
  context.stroke()
  context.textAlign = 'right'
  context.fillText(tickLabel(value), margin.left - 9, position + 4)
}
for (let index = 0; index < 6; index += 1) {
  const value = minX + ((maxX - minX) * index) / 5
  const position = x(value)
  context.beginPath()
  context.moveTo(position, margin.top)
  context.lineTo(position, height - margin.bottom)
  context.stroke()
  context.textAlign = 'center'
  context.fillText(formatElapsed(value), position, height - margin.bottom + 21)
}
context.textAlign = 'left'
context.fillText('速率（KB/s）', 8, 16)
context.textAlign = 'center'
context.fillText('运行时间', width / 2, height - 8)
const series = [
  { key: 'actualRateKbps' as const, color: '#3478f6' },
  { key: 'predictedRateKbps' as const, color: '#18a7a0' }
]
const showPointMarkers = raw.length <= 20
series.forEach((item) => {
  context.strokeStyle = item.color
  context.lineWidth = 2
  context.lineJoin = 'round'
  context.lineCap = 'round'
  context.beginPath()
  points.forEach((point, index) => {
    const xx = x(point.elapsedMillis)
    const yy = y(point[item.key])
    if (index === 0) context.moveTo(xx, yy)
    else context.lineTo(xx, yy)
```

```
      })
      context.stroke()
      if (showPointMarkers) {
        context.fillStyle = item.color
        points.forEach((point) => {
          context.beginPath()
          context.arc(x(point.elapsedMillis), y(point[item.key]), 3, 0, Math.PI * 2)
          context.fill()
        })
      }
    })
}
function nearestPoint(target: number) {
  const points = filteredPoints.value
  if (!points.length) return null
  let low = 0
  let high = points.length - 1
  while (low < high) {
    const middle = Math.floor((low + high) / 2)
    if (points[middle].elapsedMillis < target) low = middle + 1
    else high = middle
  }
  const right = points[low]
  const left = low > 0 ? points[low - 1] : right
  return Math.abs(left.elapsedMillis - target) <= Math.abs(right.elapsedMillis - target) ? left : right
}
function onMouseMove(event: MouseEvent) {
  const target = canvas.value
  if (!target || !frame) return
  const rect = target.getBoundingClientRect()
  const mouseX = event.clientX - rect.left
  if (mouseX < frame.left || mouseX > frame.right) {
    tooltipVisible.value = false
    return
  }
  const targetElapsed = frame.minX + ((mouseX - frame.left) / (frame.right - frame.left)) * (frame.maxX - frame.minX)
  const point = nearestPoint(targetElapsed)
  if (!point) return
  tooltipPoint.value = point
  tooltipLeft.value = Math.min(rect.width - 250, Math.max(6, mouseX + 14))
  tooltipTop.value = Math.max(6, event.clientY - rect.top - 64)
  tooltipVisible.value = true
}
function onMouseLeave() {
  tooltipVisible.value = false
}
watch(filteredPoints, async () => {
  await nextTick()
  draw()
}, { deep: true })
```

```
onMounted(() => {
  resizeObserver = new ResizeObserver(draw)
  if (chartWrap.value) resizeObserver.observe(chartWrap.value)
  nextTick(draw)
})
onBeforeUnmount(() => resizeObserver?.disconnect())
</script>
<template>
  <div class="network-prediction-module">
    <div class="network-chart-toolbar">
      <label>
        <span>显示范围</span>
        <select v-model.number="windowMs">
          <option :value="0">全部</option>
          <option :value="300000">最近 5 分钟</option>
          <option :value="900000">最近 15 分钟</option>
          <option :value="1800000">最近 30 分钟</option>
          <option :value="3600000">最近 1 小时</option>
        </select>
      </label>
      <span>{{ pointSummary }}</span>
    </div>
    <div ref="chartWrap" class="network-chart-wrap">
      <canvas ref="canvas" @mousemove="onMouseMove"
@mouseleave="onMouseLeave"></canvas>
      <div v-if="tooltipVisible && tooltipPoint" class="network-chart-tooltip" :style="tooltipStyle">
        <strong>运行时间 {{ formatElapsed(tooltipPoint.elapsedMillis) }}</strong>
        <span><em>实际速率</em><b>{{ formatRate(tooltipPoint.actualRateKbps)
}}</b></span>
        <span><em>预测速率</em><b>{{ formatRate(tooltipPoint.predictedRateKbps)
}}</b></span>
      </div>
    </div>
    <div class="network-chart-legend">
      <span><i class="actual"></i>实际速率</span>
      <span><i class="prediction"></i>预测速率</span>
    </div>
  </div>
</template>
<script setup lang="ts">
defineProps<{ title: string; icon?: string; rightText?: string; tone?: 'blue' | 'green' }>()
</script>
<template>
  <div class="panel-title">
    <div class="panel-title-main">
      <span v-if="icon" class="panel-title-icon" :class="tone || 'blue'">{{ icon }}</span>
      <h2>{{ title }}</h2>
    </div>
    <span v-if="rightText" class="panel-title-right">{{ rightText }}</span>
  </div>
</template>
<script setup lang="ts">
import { onBeforeUnmount, onMounted, ref } from 'vue'
```

```
import { useRouter } from 'vue-router'
import PanelTitle from '@/components/PanelTitle.vue'
import { dashboardApi } from '@/api/dashboard'
import { software } from '@/software'
import type { TransferTaskItem } from '@/types/dashboard'
const router = useRouter()
const tasks = ref<TransferTaskItem[]>([])
const message = ref('')
let timer: number | undefined
function priorityText(task: TransferTaskItem) {
  return ({ CRITICAL: '高', IMPORTANT: '中', NORMAL: '低' } as Record<string, string>)[task.priority?.priority || ''] || task.priority?.priority || '–'
}
function algorithmText(task: TransferTaskItem) {
  return String(task.currentDecision?.algorithm || 'AUTO').toUpperCase()
}
async function openTask(taskId: string) {
  try {
    await dashboardApi.selectTask(taskId)
    await router.push({ name: software === 'monitor' ? 'analysis' : 'transmission', query: { task: taskId } })
  } catch (error) {
    message.value = error instanceof Error ? error.message : '无法切换到任务进程'
  }
}
async function load() {
  try { tasks.value = await dashboardApi.transferTasks(); message.value = '' }
  catch (error) { message.value = error instanceof Error ? error.message : '无法获取传输任务队列' }
}
onMounted(async () => { await load(); timer = window.setInterval(load, 1000) })
onBeforeUnmount(() => { if (timer) window.clearInterval(timer) })
</script>
<template>
    <section class="panel task-queue-panel">
      <div v-if="message" class="toast-message error" role="status">{{ message }}</div>
      <PanelTitle title="传输任务队列" icon="◷" right-text="点击任务名查看进程" tone="blue" />
      <div class="strategy-task-table-wrap">
        <table class="strategy-task-table">
          <thead>
            <tr>
              <th>任务</th>
              <th>调度优先级</th>
              <th>策略</th>
              <th>进度</th>
              <th>打开文件</th>
              <th>打开文件夹</th>
            </tr>
          </thead>
          <tbody>
            <tr v-for="task in tasks" :key="task.id">
              <td><button class="task-name-button" type="button" @click="openTask(task.id)">{{ task.fileName
}}</button></td>
              <td>{{ priorityText(task) }}</td>
```

```
                    <td><b class="algorithm-code">{{ algorithmText(task) }}</b></td>
                    <td>
                      <div class="queue-progress"><i :style="{ width: `${Math.max(0, Math.min(100, task.progressPercent || 0))}%`
}"></i></div>
                      <span>{{ Number(task.progressPercent || 0).toFixed(1) }}%</span>
                    </td>
                    <td>
                      <a v-if="task.packageKind !== 'FOLDER'" :href="`/api/tasks/${task.id}/content/file`" target="_blank" rel="noopener">打开文
件</a>
                      <button v-else class="queue-action-disabled" type="button" disabled aria-disabled="true">打开文
件</button>
                    </td>
                    <td>
                      <a :href="`/api/tasks/${task.id}/browse`" target="_blank" rel="noopener">打开文件夹</a>
                    </td>
                  </tr>
                  <tr v-if="!tasks.length"><td colspan="6" class="queue-empty">暂无传输任务</td></tr>
                </tbody>
              </table>
            </div>
          </section>
</template>
import { createRouter, createWebHistory } from 'vue-router'
import AppShell from '@/layouts/AppShell.vue'
import TransmissionMonitorView from '@/views/TransmissionMonitorView.vue'
const analysisComponent = import.meta.env.VITE_SOFTWARE === 'monitor'
  ? () => import('@/views/NetworkMonitorSoftwareView.vue')
  : import.meta.env.VITE_SOFTWARE === 'prediction'
  ? () => import('@/views/PredictionSoftwareView.vue')
  : () => import('@/views/AdaptiveSoftwareView.vue')
import EffectDisplayView from '@/views/EffectDisplayView.vue'
import ReceiverDisplayView from '@/views/ReceiverDisplayView.vue'
const router = createRouter({
  history: createWebHistory(),
  routes: [
    {
      path: '/',
      component: AppShell,
      redirect: '/analysis',
      children: [
        { path: 'transmission', name: 'transmission', component: TransmissionMonitorView },
        { path: 'analysis', name: 'analysis', component: analysisComponent },
        { path: 'strategy', redirect: '/analysis' },
        { path: 'effect', name: 'effect', component: EffectDisplayView }
      ]
    },
    {
      path: '/receiver',
      name: 'receiver',
      component: ReceiverDisplayView
    }
  ]
})
```

```
export default router
export interface ApiResponse<T> {
  code: number
  message: string
  data: T
}
export type TaskState = 'IDLE' | 'RUNNING' | 'PAUSED' | 'FAILED' | 'COMPLETED'
export interface TaskSummary {
  id?: string
  state: TaskState
  stateText: string
  fileName: string
  fileSize: string
  fileType: string
  algorithm: string
  algorithmId: string
  level: string
  priority: string
  estimatedCompressedSize: string
  estimatedSaving: string
  retryCount?: number
  retryExhausted?: boolean
  message?: string
  editable: boolean
}
export interface PreTransmissionInfo {
  enabled: boolean
  mode: 'MESSAGE_QUEUE_STREAM' | 'API_CALLBACK' | string
  sourceProtocol?: string
  queue?: string
  frameBytes: number
  totalBytes: number
  frameCount: number
}
export interface FileOption {
  id: string
  name: string
  label: string
  size: string
  type: string
  description: string
  itemCount?: number
  packageKind?: 'FILE' | 'FOLDER'
  preTransmission?: PreTransmissionInfo
}
export interface IngressMetadata {
  version: string
  ingressMode: 'MESSAGE_QUEUE_STREAM' | 'API_CALLBACK' | string
  sourceProtocol: string
  sourceName: string
```

```
  topic?: string
  messageId?: string
  frameBytes: number
  totalBytes: number
  frameCount: number
  packageName: string
  receivedAt: string
}
export interface IngressResult {
  success: boolean
  message: string
  file: FileOption
  currentConfig: TaskConfig
  ingress: IngressMetadata
}
export interface AlgorithmOption {
  id: string
  name: string
  compressionRate: string
  description: string
}
export interface TaskConfig {
  fileId: string
  mode: 'AUTO' | 'MANUAL'
  algorithmId: string
  level: string
  priority: string
  state: TaskState
  stateText: string
}
export interface TaskOptions {
  fileOptions: FileOption[]
  algorithmOptions: AlgorithmOption[]
  levelOptions: string[]
  priorityOptions: string[]
  currentConfig: TaskConfig
  editable: boolean
}
export interface RabbitMqStatus {
  connected: boolean
  status: '已连接' | '未连接' | string
  message: string
  queue: string
  url: string
  publishedMessages: number
  consumedMessages: number
  ackedMessages: number
  consumedBytes: number
  lastPublishedAtMillis?: number
  lastConsumedAtMillis?: number
```

```
}
export interface ReceiverStatus {
  receiver: string
  online: boolean
  serviceStatus: string
  ip: string
  ipv4: string
  port: number
  receivedDir: string
  queuePreprocess: boolean
  priority: string
  preprocessThreads: number
  preprocessTimeoutMs: number
  maxQueueLength: number
  dashboardSync?: { active: boolean; updatedAtMillis?: number | null }
}
export interface DelayItem {
  name: string
  value: number
  color: string
}
export interface TrendPoint {
  time: string
  value: number
}
export interface TransmissionDashboard {
  task: TaskSummary
  sender: Record<string, string | number>
  receiver: {
    online: boolean
    remoteAddress?: string
    remoteConnected?: boolean
    remoteState?: 'CONNECTED' | 'DISCONNECTED'
    remoteMessage?: string
    services: { name: string; status: string }[]
    chunkCount: number
    processingTasks: number
    averageResponseDelay: number | null
    maxResponseDelay: number | null
  }
  route: {
    status: string
    nodes: { label: string; subLabel?: string; delay?: string; kind: string }[]
    steps: { name: string; status: 'done' | 'active' | 'waiting'; text: string }[]
    metrics: { label: string; value: string; note: string; tone: string }[]
  }
  progress: {
    value: number
    state: string
    chunkProgress: string
```

```
    sentSize: string
    totalSize: string
    totalLabel?: string
    transferRate: string
    stages: { name: string; status: string }[]
  }
  keyMetrics: { label: string; value: string; note: string; status?: string }[]
  activityLog: { title: string; detail: string; status: 'done' | 'active' | 'wait' }[]
  preview: { label: string; value: string; note: string }[]
}
export interface StrategyDashboard {
  task: TaskSummary
  compressionCurve: { ratio: number[]; totalTime: number[]; recommended: [number, number]; bestTime: number }
  networkForecast: {
    time: string[]
    predicted: number[]
    actual: number[]
    safe: number[]
    points: {
      elapsedMillis: number
      time: string
      actualRateKbps: number
      predictedRateKbps: number
      safeRateKbps: number
      predictionErrorPercent: number
    }[]
    modelName: string
    windowLabel: string
    yMax: number
    metrics: { label: string; value: string }[]
    annotations: { elapsedMillis: number; time: string; label: string; text: string; offset: number }[]
  }
  recommendation: { icon: string; label: string; value: string; extra?: string }[]
  adjustments: { id: string; time: string; title: string; operation: string; target: string }[]
}
export interface EffectDashboard {
  hasEvaluation: boolean
  snapshot?: boolean
  taskId?: string
  message?: string
  task: TaskSummary
  file?: Record<string, string>
  beforeDelay?: DelayItem[]
  afterDelay?: DelayItem[]
  network?: {
    status: string
    note: string
    metrics: { label: string; value: string }[]
    trend: TrendPoint[]
    yMax?: number
```

```
  }
  comparison?: { metric: string; before: string; after: string; change: string; direction?: 'down' | 'up' | 'none'; emphasis?: boolean }[]
  additionalDelayTitle?: string
  additionalDelayNote?: string
  additionalDelay?: { label: string; value: string; tone: string; note?: string }[]
  conclusion?: {
    improved?: boolean
    noCompression?: boolean
    intro?: string
    primaryLabel?: string
    primaryValue?: string
    primaryDescription?: string
    secondaryLabel?: string
    secondaryValue?: string
    secondaryDescription?: string
    timeSaved: string
    improvement: string
    totalBefore?: string
    totalAfter?: string
    details: string[]
    evaluatedAt: string
    result: string
  }
}
export interface TransferTaskItem {
  id: string
  fileName: string
  packageKind?: 'FILE' | 'FOLDER'
  itemCount?: number
  progressPercent: number
  state: string
  priority?: { priority?: string }
  currentDecision?: { algorithm?: string; level?: number }
}
<script setup lang="ts">
import { computed, onBeforeUnmount, onMounted, ref } from 'vue'
import DonutChart from '@/components/DonutChart.vue'
import PanelTitle from '@/components/PanelTitle.vue'
import { dashboardApi } from '@/api/dashboard'
import { softwareTitle } from '@/software'
import type { DelayItem, EffectDashboard, ReceiverStatus, TransferTaskItem, TransmissionDashboard } from '@/types/dashboard'
const transmission = ref<TransmissionDashboard | null>(null)
const effect = ref<EffectDashboard | null>(null)
const tasks = ref<TransferTaskItem[]>([])
const receiverStatus = ref<ReceiverStatus | null>(null)
const message = ref('')
const loaded = ref(false)
let timer: number | undefined
const fallbackBeforeDelay: DelayItem[] = [
  { name: "发送时延", value: 20, color: "#1769ff" },
```

```
  { name: "传输时延", value: 62, color: "#19dfff" },
  { name: "排队时延", value: 8, color: "#8cbb4d" },
  { name: "其他时延", value: 10, color: "#ff5e36" }
]
const fallbackAfterDelay: DelayItem[] = [
  { name: "发送时延", value: 12, color: "#1769ff" },
  { name: "传输时延", value: 48, color: "#19dfff" },
  { name: "排队时延", value: 6, color: "#8cbb4d" },
  { name: "压缩时延", value: 14, color: "#8a50ef" },
  { name: "解压时延", value: 9, color: "#ee59bd" },
  { name: "其他时延", value: 11, color: "#ff5e36" }
]
const fallbackStages = [
  { name: '文件压缩与封装', status: 'wait' },
  { name: '网络数据传输', status: 'wait' },
  { name: '接收与解压', status: 'wait' },
  { name: '完整性校验', status: 'wait' }
]
function firstError(results: PromiseSettledResult<unknown>[]) {
  const rejected = results.find((item) => item.status === 'rejected') as PromiseRejectedResult | undefined
  if (!rejected) return ''
  return rejected.reason instanceof Error ? rejected.reason.message : '接收端数据暂时不可用'
}
async function load() {
  const results = await Promise.allSettled([
    dashboardApi.transmission(),
    dashboardApi.effect(),
    dashboardApi.transferTasks(),
    dashboardApi.receiverStatus()
  ])
  if (results[0].status === 'fulfilled') transmission.value = results[0].value
  if (results[1].status === 'fulfilled') effect.value = results[1].value
  if (results[2].status === 'fulfilled') tasks.value = results[2].value
  if (results[3].status === 'fulfilled') receiverStatus.value = results[3].value
  message.value = firstError(results)
  loaded.value = true
}
onMounted(async () => {
  await load()
  timer = window.setInterval(load, 1000)
})
onBeforeUnmount(() => { if (timer) window.clearInterval(timer) })
const hasEvaluation = computed(() => Boolean(effect.value?.hasEvaluation))
const beforeDelay = computed(() => hasEvaluation.value && effect.value?.beforeDelay?.length ? effect.value.beforeDelay : fallbackBeforeDelay)
const afterDelay = computed(() => hasEvaluation.value && effect.value?.afterDelay?.length ? effect.value.afterDelay : fallbackAfterDelay)
const comparison = computed(() => effect.value?.comparison || [])
const progressValue = computed(() => Math.max(0, Math.min(100, Number(transmission.value?.progress.value || 0))))
const progressStages = computed(() => transmission.value?.progress.stages?.length ? transmission.value.progress.stages : fallbackStages)
const activeTasks = computed(() => tasks.value.filter((task) => ['QUEUED', 'PROFILING', 'WAITING_NETWORK', 'TRANSMITTING', 'VERIFYING', 'CACHED', 'RETRYING'].includes(String(task.state || '').toUpperCase())).length)
const receivedCount = computed(() => {
```

```
  const chunkCount = Number(transmission.value?.receiver.chunkCount || 0)
  return chunkCount > 0 ? chunkCount.toLocaleString('zh-CN') : tasks.value.length.toLocaleString('zh-CN')
})
const receiveState = computed(() => progressValue.value >= 100 ? '完成' : transmission.value?.progress.state || transmission.value?.task.stateText || '待命')
const serviceOnlineText = computed(() => receiverStatus.value?.serviceStatus || (receiverStatus.value?.online === false ? '离线' : '正常运行'))
const topServiceText = computed(() => receiverStatus.value?.online === false ? '接收服务离线' : '接收服务在线')
const timePrimaryLabel = computed(() => effect.value?.conclusion?.primaryLabel || '时间节省')
const timePrimaryValue = computed(() => effect.value?.conclusion?.primaryValue || (hasEvaluation.value ? '0 ms' : '等待结果'))
const timeSavingRate = computed(() => effect.value?.conclusion?.improvement || (hasEvaluation.value ? '0%' : '等待结果'))
const processLift = computed(() => effect.value?.conclusion?.secondaryValue || (hasEvaluation.value ? '0%' : '等待结果'))
const delayMetricRows = computed(() => {
  if (comparison.value.length) return comparison.value
  const waiting = hasEvaluation.value ? '—' : '等待结果'
  return [
    { metric: '总时延', before: waiting, after: waiting },
    { metric: '传输时延', before: '—', after: '—' },
    { metric: '排队时延', before: '—', after: '—' },
    { metric: '压缩时延', before: '—', after: '—' },
    { metric: '解压时延', before: '—', after: '—' },
    { metric: '其他时延', before: '—', after: '—' }
  ]
})
const beforeStats = computed(() => delayMetricRows.value.map((row) => ({ label: row.metric, value: row.before })))
const afterStats = computed(() => delayMetricRows.value.map((row) => ({ label: row.metric, value: row.after })))
const summaryCards = computed(() => [
  { label: '接收文件', value: receivedCount.value, tone: 'blue' },
  { label: '接收数据量', value: transmission.value?.progress.sentSize || '—', tone: 'cyan' },
  { label: '平均处理速率', value: transmission.value?.progress.transferRate || '—', tone: 'blue' },
  { label: '当前状态', value: receiveState.value, tone: progressValue.value >= 100 ? 'green' : 'blue' }
])
const configRows = computed(() => [
  { label: '服务状态', value: serviceOnlineText.value, tone: 'green' },
  { label: '当前接收端  IP (IPv4)', value: receiverStatus.value?.ipv4 || receiverStatus.value?.ip || '正在获取' },
  { label: '接收端端口', value: String(receiverStatus.value?.port || window.location.port || '8081') },
  { label: '队列预处理', value: receiverStatus.value?.queuePreprocess === false ? '停用' : '启用' },
  { label: '预处理优先级', value: receiverStatus.value?.priority || '中' },
  { label: '预处理线程数', value: String(receiverStatus.value?.preprocessThreads || 4) },
  { label: '预处理超时  (ms)', value: String(receiverStatus.value?.preprocessTimeoutMs || 5000) },
  { label: '最大队列长度', value: String(receiverStatus.value?.maxQueueLength || 500) }
])
function priorityText(task: TransferTaskItem) {
  const priority = task.priority?.priority || ''
  return ({ CRITICAL: '高', IMPORTANT: '中', NORMAL: '低' } as Record<string, string>)[priority] || priority || '–'
}
function algorithmText(task: TransferTaskItem) {
  return String(task.currentDecision?.algorithm || 'AUTO').toUpperCase()
}
function progressPercent(value?: number) {
  return Math.max(0, Math.min(100, Number(value) || 0))
}
```

```
function progressText(value?: number) {
  const safeValue = progressPercent(value)
  return safeValue >= 100 ? '100%' : safeValue.toFixed(1) + '%'
}
function stageMark(status: string) {
  if (status === 'done') return '√'
  if (status === 'active') return '●'
  return '○'
}
</script>
<template>
  <div class="receiver-display-shell">
    <header class="topbar">
      <div class="brand">
        <span class="brand-cube"><i></i><i></i><i></i><i></i></span>
        <span>{{ softwareTitle }}</span>
      </div>
      <div class="top-actions">
        <span class="top-status"><b class="status-dot"></b>系统在线</span>
        <span class="top-divider"></span>
        <span class="top-status muted">{{ topServiceText }}</span>
      </div>
    </header>
    <aside class="receiver-display-sidebar">
      <div class="collapse-mark">☰</div>
      <nav>
        <span class="nav-item active"><span class="nav-icon">↙</span><span>接收监
控</span></span>
      </nav>
    </aside>
    <main class="receiver-display-area">
      <div v-if="message" class="toast-message error">{{ message }}</div>
      <section class="receiver-display-page" :class="{ loading: !loaded }">
        <section class="panel receiver-time-panel">
          <PanelTitle title="压缩前后时间对比" icon="▣" />
          <div class="receiver-time-compare">
            <article class="receiver-time-card">
              <header>压缩前</header>
              <div class="receiver-time-body">
                <DonutChart :data="beforeDelay"    :height="210" />
                <div class="receiver-time-stats">
                  <div v-for="item in beforeStats" :key="item.label">
                    <span>{{ item.label }}</span>
                    <b>{{ item.value }}</b>
                  </div>
                </div>
              </div>
              <footer>
                <span v-for="item in beforeDelay" :key="item.name"><i :style="{ background: item.color }"></i>{{ item.name
}}</span>
              </footer>
            </article>
```

```
            <div class="receiver-time-arrows"><span>→</span><span>→</span><span>→</
span></div>
            <article class="receiver-time-card after">
              <header>压缩后</header>
              <div class="receiver-time-body">
                <DonutChart :data="afterDelay"    :height="210" />
                <div class="receiver-time-stats">
                  <div v-for="item in afterStats" :key="item.label">
                    <span>{{ item.label }}</span>
                    <b>{{ item.value }}</b>
                  </div>
                </div>
              </div>
              <footer>
                <span v-for="item in afterDelay" :key="item.name"><i :style="{ background: item.color }"></i>{{ item.name
}}</span>
              </footer>
            </article>
          </div>
          <div class="receiver-time-saving-row">
            <article><span>{{ timePrimaryLabel }}</span><b>{{ timePrimaryValue }}</b></article>
            <article><span>时间节省率</span><b class="green">{{ timeSavingRate }}</b></article>
            <article><span>处理效率提升</span><b>{{ processLift }}</b></article>
          </div>
        </section>
        <section class="panel receiver-config-panel">
          <PanelTitle title="接收端配置" icon="◎" />
          <div class="receiver-config-list">
            <div v-for="row in configRows" :key="row.label" class="receiver-config-row">
              <i :class="row.tone || ''"></i>
              <span>{{ row.label }}</span>
              <b :class="row.tone || ''">{{ row.value }}</b>
            </div>
          </div>
        </section>
        <section class="panel receiver-summary-panel">
          <PanelTitle title="接收总结" icon="▥" />
          <div class="receiver-summary-cards">
            <article v-for="card in summaryCards" :key="card.label" :class="card.tone">
              <span>{{ card.label }}</span>
              <b>{{ card.value }}</b>
            </article>
          </div>
          <div class="receiver-progress-block">
            <div class="receiver-progress-title"><span>传输进度</span><b>{{ progressValue.toFixed(0) }}%</b></div>
            <div class="receiver-receive-track"><i :style="{ width: progressValue + '%' }"></i></div>
            <div class="receiver-stage-row">
              <span v-for="stage in progressStages" :key="stage.name" :class="stage.status">
                <b>{{ stageMark(stage.status) }}</b>{{ stage.name }}
              </span>
            </div>
          </div>
```

```
        </section>
        <section class="panel receiver-task-panel task-queue-panel">
          <PanelTitle title="传输任务队列" icon="⏲" :right-text="'共 ' + tasks.length + ' 个任务'" tone="blue" />
          <div class="strategy-task-table-wrap receiver-task-table-wrap">
            <table class="strategy-task-table">
              <thead>
                <tr>
                  <th>任务</th>
                  <th>调度优先级</th>
                  <th>策略</th>
                  <th>进度</th>
                  <th>打开文件</th>
                  <th>打开文件夹</th>
                </tr>
              </thead>
              <tbody>
                <tr v-for="task in tasks" :key="task.id">
                  <td><span class="task-name-button receiver-task-name">{{ task.fileName
}}</span></td>
                  <td>{{ priorityText(task) }}</td>
                  <td><b class="algorithm-code">{{ algorithmText(task) }}</b></td>
                  <td>
                    <div class="queue-progress"><i :style="{ width: progressPercent(task.progressPercent) + '%'
}"></i></div>
                    <span>{{ progressText(task.progressPercent) }}</span>
                  </td>
                  <td>
                    <a
                      v-if="task.packageKind !== 'FOLDER'"
                      :href="`/api/receiver/tasks/${encodeURIComponent(task.id)}/content/
file`"
                      target="_blank"
                      rel="noopener"
                    >打开文件</a>
                    <button v-else class="queue-action-disabled" type="button" disabled aria-disabled="true">打开文
件</button>
                  </td>
                  <td>
                    <a

:href="`/api/receiver/tasks/${encodeURIComponent(task.id)}/browse`"
                      target="_blank"
                      rel="noopener"
                    >打开文件夹</a>
                  </td>
                </tr>
                <tr v-if="!tasks.length"><td colspan="6" class="queue-empty">暂无传输任务</td></tr>
              </tbody>
            </table>
          </div>
        </section>
      </section>
    </main>
  </div>
</template>
```

```
package com.datacompress.control.algorithm;
import org.springframework.stereotype.Component;
import java.util.*;
@Component
public class AlgorithmRegistry {
    private final Map<String, CompressionAlgorithm> algorithms = new
LinkedHashMap<>();
    public AlgorithmRegistry(List<CompressionAlgorithm> implementations) {
        implementations.stream()
                .sorted(Comparator.comparing(CompressionAlgorithm::name))
                .forEach(a -> algorithms.put(a.name(), a));
    }
    public CompressionAlgorithm require(String name) {
        CompressionAlgorithm value = algorithms.get(
                name == null ? "" : name.trim().toUpperCase(Locale.ROOT));
        if (value == null) throw new IllegalArgumentException("Unsupported algorithm: " + name);
        return value;
    }
    public List<CompressionAlgorithm> all() { return List.copyOf(algorithms.values()); }
}
package com.datacompress.control.algorithm;
import java.util.List;
public abstract class BaseAlgorithm implements CompressionAlgorithm {
    private final String name;
    private final boolean levels;
    private final int min;
    private final int max;
    private final int defaultLevel;
    private final List<Integer> realtimeLevels;
    protected BaseAlgorithm(String name, boolean levels, int min, int max,
                            int defaultLevel, List<Integer> realtimeLevels) {
        this.name = name;
        this.levels = levels;
        this.min = min;
        this.max = max;
        this.defaultLevel = defaultLevel;
        this.realtimeLevels = List.copyOf(realtimeLevels);
    }
    public String name() { return name; }
    public boolean supportsLevel() { return levels; }
    public int minLevel() { return min; }
    public int maxLevel() { return max; }
    public int defaultLevel() { return defaultLevel; }
    public List<Integer> realtimeLevels() { return realtimeLevels; }
}
package com.datacompress.control.algorithm;
import com.aayushatharva.brotli4j.Brotli4jLoader;
import com.aayushatharva.brotli4j.decoder.*;
import com.aayushatharva.brotli4j.encoder.Encoder;
import org.springframework.stereotype.Component;
import java.util.List;
```

```
@Component
public class BrotliAlgorithm extends BaseAlgorithm {
    public BrotliAlgorithm() { super("BROTLI", true, 0, 11, 5, List.of(3, 6)); }
    public byte[] compress(byte[] input, int level) throws Exception {
        Brotli4jLoader.ensureAvailability();
        return Encoder.compress(input, Encoder.Parameters.create(normalizeLevel(level)));
    }
    public byte[] decompress(byte[] input) throws Exception {
        Brotli4jLoader.ensureAvailability();
        DirectDecompress result = Decoder.decompress(input);
        if (result.getResultStatus() != DecoderJNI.Status.DONE) {
            throw new IllegalStateException("Brotli status: " + result.getResultStatus());
        }
        return result.getDecompressedData();
    }
}
package com.datacompress.control.algorithm;
import org.apache.commons.compress.compressors.bzip2.*;
import org.springframework.stereotype.Component;
import java.io.*;
import java.util.List;
@Component
public class Bzip2Algorithm extends BaseAlgorithm {
    public Bzip2Algorithm() { super("BZIP2", true, 1, 9, 6, List.of(3, 6)); }
    public byte[] compress(byte[] input, int level) throws Exception {
        ByteArrayOutputStream out = new ByteArrayOutputStream();
        try (BZip2CompressorOutputStream stream =
                    new BZip2CompressorOutputStream(out, normalizeLevel(level)))
{
            stream.write(input);
        }
        return out.toByteArray();
    }
    public byte[] decompress(byte[] input) throws Exception {
        try (BZip2CompressorInputStream in =
                    new BZip2CompressorInputStream(new
ByteArrayInputStream(input))) {
            return in.readAllBytes();
        }
    }
}
package com.datacompress.control.algorithm;
import java.util.List;
public interface CompressionAlgorithm {
    String name();
    boolean supportsLevel();
    int minLevel();
    int maxLevel();
    int defaultLevel();
    List<Integer> realtimeLevels();
    byte[] compress(byte[] input, int level) throws Exception;
    byte[] decompress(byte[] input) throws Exception;
```

```
    default int normalizeLevel(int level) {
        return supportsLevel()
                ? Math.max(minLevel(), Math.min(maxLevel(), level))
                : defaultLevel();
    }
}
package com.datacompress.control.algorithm;
import org.springframework.stereotype.Component;
import java.io.*;
import java.util.List;
import java.util.zip.*;
@Component
public class DeflateAlgorithm extends BaseAlgorithm {
    public DeflateAlgorithm() { super("DEFLATE", true, 0, 9, 6, List.of(1, 6)); }
    public byte[] compress(byte[] input, int level) throws Exception {
        ByteArrayOutputStream out = new ByteArrayOutputStream();
        Deflater deflater = new Deflater(normalizeLevel(level));
        try (DeflaterOutputStream stream = new DeflaterOutputStream(out, deflater)) {
            stream.write(input);
        } finally {
            deflater.end();
        }
        return out.toByteArray();
    }
    public byte[] decompress(byte[] input) throws Exception {
        try (InflaterInputStream in = new InflaterInputStream(new ByteArrayInputStream(input))) {
            return in.readAllBytes();
        }
    }
}
package com.datacompress.control.algorithm;
import org.springframework.stereotype.Component;
import java.io.*;
import java.util.List;
import java.util.zip.*;
@Component
public class GzipAlgorithm extends BaseAlgorithm {
    public GzipAlgorithm() { super("GZIP", true, 0, 9, 6, List.of(1, 6)); }
    public byte[] compress(byte[] input, int level) throws Exception {
        ByteArrayOutputStream out = new ByteArrayOutputStream();
        try (LevelGzip stream = new LevelGzip(out, normalizeLevel(level))) {
            stream.write(input);
        }
        return out.toByteArray();
    }
    public byte[] decompress(byte[] input) throws Exception {
        try (GZIPInputStream in = new GZIPInputStream(new ByteArrayInputStream(input))) {
            return in.readAllBytes();
        }
    }
```

```
    private static final class LevelGzip extends GZIPOutputStream {
        LevelGzip(OutputStream out, int level) throws IOException {
            super(out);
            def.setLevel(level);
        }
    }
}
package com.datacompress.control.algorithm;
import net.jpountz.lz4.*;
import org.springframework.stereotype.Component;
import java.nio.ByteBuffer;
import java.util.List;
@Component
public class Lz4Algorithm extends BaseAlgorithm {
    private final LZ4Factory factory = LZ4Factory.fastestInstance();
    public Lz4Algorithm() { super("LZ4", true, 0, 9, 0, List.of(0, 6)); }
    public byte[] compress(byte[] input, int level) {
        LZ4Compressor compressor = normalizeLevel(level) == 0
                ? factory.fastCompressor()
                : factory.highCompressor(normalizeLevel(level));
        byte[] buffer = new byte[compressor.maxCompressedLength(input.length)];
        int n = compressor.compress(input, 0, input.length, buffer, 0, buffer.length);
        ByteBuffer frame = ByteBuffer.allocate(8 + n);
        frame.putInt(input.length).putInt(n).put(buffer, 0, n);
        return frame.array();
    }
    public byte[] decompress(byte[] input) {
        ByteBuffer frame = ByteBuffer.wrap(input);
        int original = frame.getInt();
        int compressed = frame.getInt();
        byte[] data = new byte[compressed];
        frame.get(data);
        byte[] restored = new byte[original];
        factory.fastDecompressor().decompress(data, 0, restored, 0, original);
        return restored;
    }
}
package com.datacompress.control.algorithm;
import org.springframework.stereotype.Component;
import org.tukaani.xz.*;
import java.io.*;
import java.util.List;
@Component
public class LzmaAlgorithm extends BaseAlgorithm {
    public LzmaAlgorithm() { super("LZMA", true, 0, 9, 4, List.of(1, 4)); }
    public byte[] compress(byte[] input, int level) throws Exception {
        ByteArrayOutputStream out = new ByteArrayOutputStream();
        try (XZOutputStream stream =
                     new XZOutputStream(out, new
LZMA2Options(normalizeLevel(level)))) {
            stream.write(input);
```

```
            }
            return out.toByteArray();
        }
        public byte[] decompress(byte[] input) throws Exception {
            try (XZInputStream in = new XZInputStream(new ByteArrayInputStream(input))) {
                return in.readAllBytes();
            }
        }
}
package com.datacompress.control.algorithm;
import org.springframework.stereotype.Component;
import java.util.List;
@Component
public class NoneAlgorithm extends BaseAlgorithm {
    public NoneAlgorithm() { super("NONE", false, 0, 0, 0, List.of(0)); }
    public byte[] compress(byte[] input, int level) { return input.clone(); }
    public byte[] decompress(byte[] input) { return input.clone(); }
}
package com.datacompress.control.algorithm;
import org.springframework.stereotype.Component;
import org.xerial.snappy.Snappy;
import java.util.List;
@Component
public class SnappyAlgorithm extends BaseAlgorithm {
    public SnappyAlgorithm() { super("SNAPPY", false, 0, 0, 0, List.of(0)); }
    public byte[] compress(byte[] input, int level) throws Exception { return Snappy.compress(input); }
    public byte[] decompress(byte[] input) throws Exception { return Snappy.uncompress(input); }
}
package com.datacompress.control.algorithm;
import org.springframework.stereotype.Component;
import java.io.*;
import java.util.List;
import java.util.zip.*;
@Component
public class ZipAlgorithm extends BaseAlgorithm {
    public ZipAlgorithm() { super("ZIP", true, 0, 9, 6, List.of(1, 6)); }
    public byte[] compress(byte[] input, int level) throws Exception {
        ByteArrayOutputStream out = new ByteArrayOutputStream();
        try (ZipOutputStream zip = new ZipOutputStream(out)) {
            zip.setLevel(normalizeLevel(level));
            zip.putNextEntry(new ZipEntry("payload.bin"));
            zip.write(input);
            zip.closeEntry();
        }
        return out.toByteArray();
    }
    public byte[] decompress(byte[] input) throws Exception {
        try (ZipInputStream zip = new ZipInputStream(new ByteArrayInputStream(input))) {
            if (zip.getNextEntry() == null) throw new IllegalArgumentException("ZIP entry missing");
            return zip.readAllBytes();
```

```
            }
        }
}
package com.datacompress.control.algorithm;
import com.github.luben.zstd.Zstd;
import org.springframework.stereotype.Component;
import java.nio.ByteBuffer;
import java.util.List;
@Component
public class ZstdAlgorithm extends BaseAlgorithm {
    public ZstdAlgorithm() { super("ZSTD", true, 1, 22, 3, List.of(1, 3, 6)); }
    public byte[] compress(byte[] input, int level) {
        byte[] compressed = Zstd.compress(input, normalizeLevel(level));
        return ByteBuffer.allocate(4 + compressed.length)
                .putInt(input.length).put(compressed).array();
    }
    public byte[] decompress(byte[] input) {
        ByteBuffer frame = ByteBuffer.wrap(input);
        int original = frame.getInt();
        byte[] compressed = new byte[frame.remaining()];
        frame.get(compressed);
        byte[] restored = new byte[original];
        long result = Zstd.decompress(restored, compressed);
        if (Zstd.isError(result)) throw new IllegalStateException(Zstd.getErrorName(result));
        return restored;
    }
}
package com.datacompress.control.controller;
import com.datacompress.control.domain.AlarmRecord;
import com.datacompress.control.service.AlarmService;
import org.springframework.web.bind.annotation.*;
import java.util.List;
import java.util.Map;
@RestController
@RequestMapping("/api/alarms")
public class AlarmController {
    private final AlarmService service;
    public AlarmController(AlarmService service) { this.service = service; }
    @GetMapping
    public List<AlarmRecord> list(@RequestParam(value = "level", required = false) String level,
                                  @RequestParam(value = "status", required = false) String
status) {
        return service.list(level, status);
    }
    @PostMapping("/{id}/resolve")
    public AlarmRecord resolve(@PathVariable("id") String id,
                               @RequestBody(required = false) Map<String,String>
body) {
        return service.resolve(id, body == null ? "" : body.getOrDefault("note", ""));
    }
}
package com.datacompress.control.controller;
```

```
import com.datacompress.control.service.*;
import org.springframework.web.bind.annotation.*;
import java.util.*;
@RestController
@RequestMapping("/api/dashboard")
public class DashboardController {
    private final TransferTaskService tasks;
    private final DataSourceService sources;
    private final NetworkTelemetryService network;
    private final AlarmService alarms;
    public DashboardController(TransferTaskService tasks, DataSourceService sources,
                               NetworkTelemetryService network, AlarmService
alarms) {
        this.tasks = tasks;
        this.sources = sources;
        this.network = network;
        this.alarms = alarms;
    }
    @GetMapping
    public Map<String,Object> snapshot() {
        Map<String,Object> result = new LinkedHashMap<>();
        result.put("network", network.current());
        result.put("sourceCount", sources.list().size());
        result.put("onlineSourceCount", sources.list().stream().filter(s -> s.online()).count());
        result.put("queueSize", tasks.queueSize());
        result.put("queuedBytes", tasks.queuedBytes());
        result.put("completedCount", tasks.completedCount());
        result.put("failedCount", tasks.failedCount());
        result.put("openAlarmCount", alarms.openCount());
        return result;
    }
}
package com.datacompress.control.controller;
import com.datacompress.control.domain.DataSourceInfo;
import com.datacompress.control.service.DataSourceService;
import org.springframework.web.bind.annotation.*;
import java.util.List;
@RestController
@RequestMapping("/api/sources")
public class DataSourceController {
    private final DataSourceService service;
    public DataSourceController(DataSourceService service) { this.service = service; }
    @GetMapping public List<DataSourceInfo> list() { return service.list(); }
    @PostMapping public DataSourceInfo save(@RequestBody DataSourceInfo source)
{
        return service.save(source);
    }
}
package com.datacompress.control.controller;
import org.springframework.http.*;
import org.springframework.web.bind.annotation.*;
import java.time.Instant;
```

```
import java.util.Map;
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<Map<String,Object>> badRequest(IllegalArgumentException e) {
        return ResponseEntity.badRequest().body(Map.of(
                "timestamp", Instant.now(), "status", 400, "message", safe(e)));
    }
    @ExceptionHandler(Throwable.class)
    public ResponseEntity<Map<String,Object>> error(Throwable e) {
        return
ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(Map.of(
                "timestamp", Instant.now(), "status", 500, "message", safe(e)));
    }
    private String safe(Throwable e) {
        return e.getMessage() == null ? e.getClass().getSimpleName() : e.getMessage();
    }
}
package com.datacompress.control.controller;
import com.datacompress.control.domain.*;
import com.datacompress.control.service.*;
import org.springframework.web.bind.annotation.*;
import java.util.*;
@RestController
@RequestMapping("/api/network")
public class NetworkController {
    private final NetworkTelemetryService network;
    private final CompactLstmRatePredictor predictor;
    public NetworkController(NetworkTelemetryService network,
                             CompactLstmRatePredictor predictor) {
        this.network = network;
        this.predictor = predictor;
    }
    @GetMapping("/current")
    public NetworkSample current() { return network.current(); }
    @GetMapping("/history")
    public List<NetworkSample> history(@RequestParam(value = "limit", defaultValue = "120") int limit) {
        return network.history(limit);
    }
    @GetMapping("/forecast")
    public RateForecast forecast() { return predictor.predictCurrent(); }
    @GetMapping("/health")
    public Map<String,Object> health() {
        Map<String,Object> result = new LinkedHashMap<>();
        result.put("telemetry", network.status());
        result.put("prediction", predictor.health());
        result.put("measurementMode", network.isSimulationEnabled()
                ? "SIMULATION_COMPATIBILITY" :
"EXTERNAL_OR_INTERNAL_MEASUREMENT");
        return result;
    }
    @PostMapping("/telemetry")
```

```
    public NetworkSample ingest(@RequestBody NetworkSample sample) {
        network.ingest(sample);
        return network.current();
    }
    @PostMapping("/simulation")
    public Map<String,Object> simulation(@RequestParam("enabled") boolean enabled) {
        network.setSimulationEnabled(enabled);
        return Map.of("simulationEnabled", network.isSimulationEnabled());
    }
}
package com.datacompress.control.domain;
public enum AlarmLevel { INFO, WARNING, MAJOR, CRITICAL }
package com.datacompress.control.domain;
import java.time.Instant;
public record AlarmRecord(
        String id,
        AlarmLevel level,
        AlarmType type,
        String source,
        Instant occurredAt,
        String affectedTaskId,
        String description,
        String suggestion,
        String status,
        String handlingNote) {
}
package com.datacompress.control.domain;
public enum AlarmType { LOW_BANDWIDTH, HIGH_RTT, HIGH_JITTER, HIGH_PACKET_LOSS, DISCONNECTED, QUEUE_BACKLOG, TRANSFER_FAILED, RETRY_EXHAUSTED, DECOMPRESSION_FAILED,
INTEGRITY_FAILED, RESOURCE_SHORTAGE }
package com.datacompress.control.domain;
public record AlgorithmProfile(
        String algorithm,
        int level,
        boolean available,
        double compressedRatio,
        double compressionNanosPerByte,
        double decompressionNanosPerByte,
        String error) {
    public long compressedBytes(long originalBytes) {
        return "NONE".equals(algorithm)
                ? originalBytes
                : Math.max(1L, Math.round(originalBytes * compressedRatio));
    }
    public double compressionMillis(long bytes) {
        return compressionNanosPerByte * bytes / 1_000_000.0D;
    }
    public double decompressionMillis(long bytes) {
        return decompressionNanosPerByte * bytes / 1_000_000.0D;
    }
}
package com.datacompress.control.domain;
```

```
public record ChunkMetric(
        int chunkIndex,
        long offsetBytes,
        int originalBytes,
        int compressedBytes,
        String algorithm,
        int level,
        int chunkSizeBytes,
        double compressionMillis,
        double modeledTransferMillis,
        double decompressionMillis,
        double actualRateKbps,
        double predictedRateKbps,
        boolean checksumPassed,
        int retries,
        long elapsedMillis) {
}
package com.datacompress.control.domain;
public record CompressionDecision(
        String preprocess,
        String algorithm,
        int level,
        int chunkSizeBytes,
        boolean cache,
        boolean batch,
        int sendOrder,
        double estimatedCompressionMillis,
        double estimatedTransferMillis,
        double estimatedDecompressionMillis,
        double estimatedTotalMillis,
        double estimatedCompressedRatio,
        String reason) {
    public String strategyKey() {
        return algorithm + "-" + level + "-" + chunkSizeBytes;
    }
}
package com.datacompress.control.domain;
public enum ConnectionState { CONNECTED, UNSTABLE, CONGESTED, DISCONNECTED,
RECOVERING }
package com.datacompress.control.domain;
public enum DataModality { NUMERIC_TIME_SERIES, DEVICE_OPERATION, TEXT_LOG, IMAGE_VIDEO, STATISTICAL_REPORT, ALARM_EVENT, CONTROL_MESSAGE,
PACKAGE_DATA, OTHER_FILE }
package com.datacompress.control.domain;
public enum DataPriority { CRITICAL, IMPORTANT, NORMAL }
package com.datacompress.control.domain;
import java.time.Instant;
public record DataSourceInfo(
        String id,
        String name,
        String type,
        String endpoint,
        String deviceCode,
```

```
        String businessScene,
        boolean online,
        Instant lastReportTime,
        long cumulativeBytes,
        int collectionPeriodMs) {
}
package com.datacompress.control.domain;
public record NetworkCurvePoint(
        long elapsedMillis,
        double actualRateKbps,
        double predictedRateKbps,
        double safeRateKbps,
        double predictionErrorPercent,
        String strategy) {
}
package com.datacompress.control.domain;
public enum NetworkQuality { GOOD, FLUCTUATING, CONGESTED, DISCONNECTED,
RECOVERING }
package com.datacompress.control.domain;
import java.time.Instant;
public record NetworkSample(
        Instant timestamp,
        double throughputKbps,
        double rttMillis,
        double jitterMillis,
        double packetLossPercent,
        int retransmissions,
        ConnectionState connectionState,
        NetworkQuality quality,
        long queuedBytes,
        int cachedTasks,
        double cpuPercent,
        double memoryPercent,
        boolean externallyMeasured) {
}
package com.datacompress.control.domain;
import java.time.Instant;
public record PredictionHealth(
        Instant checkedAt,
        boolean modelAllowed,
        boolean degraded,
        String reason,
        int rollingSamples,
        double rollingMaeKbps,
        double rollingMapePercent,
        double unsafePredictionRatioPercent,
        double correctionFactor,
        long cooldownRemainingMillis) {
}
package com.datacompress.control.domain;
public record PriorityDecision(
```

```
        DataPriority priority,
        int priorityValue,
        long deadlineEpochMs,
        long maxWaitMillis,
        boolean cacheAllowed,
        boolean batchAllowed,
        String recommendedCompressionPolicy,
        String failurePolicy,
        String reason) {
}
package com.datacompress.control.domain;
import java.time.Instant;
import java.util.List;
public record RateForecast(
        Instant generatedAt,
        List<Double> predictedRatesKbps,
        double predictedAverageKbps,
        double safeRateKbps,
        double recentMaeKbps,
        double recentMapePercent,
        double correctionFactor,
        String modelName,
        long inferenceMillis,
        boolean telemetryFresh,
        long telemetryAgeMillis,
        boolean degraded,
        String degradationReason,
        double baselineP20Kbps,
        double shortAverageKbps,
        double mediumAverageKbps,
        double unsafePredictionRatioPercent,
        int feedbackSamples) {
    public RateForecast {
        predictedRatesKbps = List.copyOf(predictedRatesKbps);
    }
}
package com.datacompress.control.domain;
import java.util.List;
public record RecognitionResult(
        DataModality modality,
        String format,
        String mimeType,
        boolean alreadyCompressed,
        double compressibilityScore,
        String suggestedPreprocess,
        List<String> candidateAlgorithms,
        String recommendedLevelRange,
        double confidence,
        String strategyTag,
        String reason) {
```

```
    public RecognitionResult {
        candidateAlgorithms = List.copyOf(candidateAlgorithms);
    }
}
package com.datacompress.control.domain;
public record StrategyAdjustment(
        long elapsedMillis,
        String fromStrategy,
        String toStrategy,
        double actualRateKbps,
        double predictedRateKbps,
        double predictionErrorPercent,
        double estimatedGainPercent,
        String reason) {
}
package com.datacompress.control.domain;
public enum TaskState { QUEUED, PROFILING, WAITING_NETWORK, TRANSMITTING, VERIFYING, COMPLETED, FAILED, CACHED, RETRYING,
PAUSED, CANCELLED }
package com.datacompress.control.domain;
import java.time.Instant;
public record TelemetryStatus(
        Instant checkedAt,
        boolean fresh,
        long ageMillis,
        String source,
        double conservativeRateKbps,
        double p20OneMinuteKbps,
        double averageOneMinuteKbps,
        double averageFiveMinutesKbps,
        double averageOneHourKbps,
        int retainedSamples) {
}
package com.datacompress.control.domain;
import java.util.List;
public record TransferTaskView(
        String id,
        String fileName,
        String packageKind,
        int itemCount,
        String sourceId,
        String businessScene,
        long originalBytes,
        long compressedBytes,
        double progressPercent,
        TaskState state,
        RecognitionResult recognition,
        PriorityDecision priority,
        CompressionDecision currentDecision,
        int totalChunks,
        int completedChunks,
        int retryCount,
```

```
            long queuedAtMillis,
            long startedAtMillis,
            long completedAtMillis,
            double totalCompressionMillis,
            double totalTransferMillis,
            double totalDecompressionMillis,
            boolean integrityPassed,
            String checksum,
            String message,
            List<NetworkCurvePoint> networkCurve,
            List<StrategyAdjustment> adjustments,
            List<ChunkMetric> chunks) {
        public TransferTaskView {
            packageKind = "FOLDER".equalsIgnoreCase(packageKind) ? "FOLDER" : "FILE";
            itemCount = Math.max(1, itemCount);
            networkCurve = networkCurve == null ? List.of() : List.copyOf(networkCurve);
            adjustments = adjustments == null ? List.of() : List.copyOf(adjustments);
            chunks = chunks == null ? List.of() : List.copyOf(chunks);
        }
}
package com.datacompress.control.service;
import com.datacompress.control.domain.*;
import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
@Service
public class AlarmService {
    private final Map<String, AlarmRecord> alarms = new
ConcurrentHashMap<>();
    private final Map<String, Long> cooldown = new ConcurrentHashMap<>();
    private static final long NETWORK_ALARM_COOLDOWN_MS = 60_000L;
    public AlarmRecord raise(AlarmLevel level, AlarmType type, String source,
                             String taskId, String description, String suggestion) {
        String key = (source == null ? "" : source) + ":" + type.name()
                + ":" + (taskId == null ? "" : taskId);
        Optional<AlarmRecord> existingOpen = alarms.values().stream()
                .filter(a -> "OPEN".equals(a.status()))
                .filter(a -> a.type() == type)
                .filter(a -> Objects.equals(a.source(), source))
                .filter(a -> Objects.equals(a.affectedTaskId(), taskId))
                .findFirst();
        if (existingOpen.isPresent()) return existingOpen.get();
        long now = System.currentTimeMillis();
        Long last = cooldown.get(key);
        if (last != null && now - last < NETWORK_ALARM_COOLDOWN_MS) {
            return alarms.values().stream()
                    .filter(a -> a.type() == type)
                    .filter(a -> Objects.equals(a.source(), source))
                    .filter(a -> Objects.equals(a.affectedTaskId(), taskId))
                    .max(Comparator.comparing(AlarmRecord::occurredAt))
```

```
                .orElseGet(() -> create(level, type, source, taskId, description, suggestion, key, now));
        }
        return create(level, type, source, taskId, description, suggestion, key, now);
    }
    private AlarmRecord create(AlarmLevel level, AlarmType type, String source,
                               String taskId, String description, String suggestion,
                               String key, long now) {
        AlarmRecord record = new AlarmRecord(UUID.randomUUID().toString(), level,
type,
                source, Instant.now(), taskId, description, suggestion, "OPEN", "");
        alarms.put(record.id(), record);
        cooldown.put(key, now);
        return record;
    }
    public List<AlarmRecord> list(String level, String status) {
        return alarms.values().stream()
                .filter(a -> level == null || level.isBlank() || a.level().name().equalsIgnoreCase(level))
                .filter(a -> status == null || status.isBlank() || a.status().equalsIgnoreCase(status))
                .sorted(Comparator.comparing(AlarmRecord::occurredAt).reversed())
                .toList();
    }
    public AlarmRecord resolve(String id, String note) {
        AlarmRecord old = Optional.ofNullable(alarms.get(id))
                .orElseThrow(() -> new IllegalArgumentException("告警不存在"));
        AlarmRecord updated = new AlarmRecord(old.id(), old.level(), old.type(), old.source(),
                old.occurredAt(), old.affectedTaskId(), old.description(), old.suggestion(),
                "RESOLVED", note == null ? "" : note);
        alarms.put(id, updated);
        return updated;
    }
    public long openCount() {
        return alarms.values().stream().filter(a -> "OPEN".equals(a.status())).count();
    }
}
package com.datacompress.control.service;
import com.datacompress.control.algorithm.*;
import com.datacompress.control.domain.*;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
@Service
public class CompressionProfileService {
    private static final int CRITICAL_NO_COMPRESSION_MAX_BYTES = 16 * 1024 *
1024;
    private final AlgorithmRegistry registry;
    private final int sampleBytes;
    private final Map<String, List<AlgorithmProfile>> cache = new ConcurrentHashMap<>();
    public CompressionProfileService(
            AlgorithmRegistry registry,
            @Value("${app.strategy.profile-sample-bytes:65536}") int sampleBytes) {
        this.registry = registry;
```

```
        this.sampleBytes = sampleBytes;
    }
    public List<AlgorithmProfile> profile(byte[] data, RecognitionResult recognition,
                                          PriorityDecision priority, String mode,
                                          String manualAlgorithm, Integer
manualLevel) {
        String signature = recognition.modality() + ":" + data.length + ":"
                + Arrays.hashCode(Arrays.copyOf(data, Math.min(1024, data.length)))
                + ":" + mode + ":" + manualAlgorithm + ":" + manualLevel;
        return cache.computeIfAbsent(signature,
                key -> build(data, recognition, priority, mode, manualAlgorithm, manualLevel));
    }
    private List<AlgorithmProfile> build(byte[] data, RecognitionResult recognition,
                                         PriorityDecision priority, String mode,
                                         String manualAlgorithm, Integer
manualLevel) {
        byte[] sample = representativeSample(data);
        List<Candidate> candidates = candidates(recognition, priority, mode, manualAlgorithm, manualLevel, data.length);
        List<AlgorithmProfile> result = new ArrayList<>();
        for (Candidate candidate : candidates) {
            CompressionAlgorithm algorithm = registry.require(candidate.algorithm());
            int level = algorithm.normalizeLevel(candidate.level());
            try {
                long c0 = System.nanoTime();
                byte[] compressed = algorithm.compress(sample, level);
                long cNanos = System.nanoTime() - c0;
                long d0 = System.nanoTime();
                byte[] restored = algorithm.decompress(compressed);
                long dNanos = System.nanoTime() - d0;
                if (!Arrays.equals(sample, restored)) {
                    throw new IllegalStateException("解压结果不一致");
                }
                double bytes = Math.max(1.0, sample.length);
                result.add(new AlgorithmProfile(algorithm.name(), level, true,
                        compressed.length / bytes, cNanos / bytes, dNanos / bytes, null));
            } catch (Throwable error) {
                result.add(new AlgorithmProfile(algorithm.name(), level, false,
                        1.0, 0, 0, error.getClass().getSimpleName() + ": " + error.getMessage()));
            }
        }
        return result;
    }
    private List<Candidate> candidates(RecognitionResult recognition,
                                       PriorityDecision priority, String mode,
                                       String manualAlgorithm, Integer
manualLevel,
                                       int dataLength) {
        if ("MANUAL".equalsIgnoreCase(mode)) {
            String algorithm = manualAlgorithm == null ? "ZSTD" : manualAlgorithm;
            CompressionAlgorithm value = registry.require(algorithm);
            return List.of(new Candidate(value.name(),
                    manualLevel == null ? value.defaultLevel() : manualLevel));
        }
```

```
        if (recognition.alreadyCompressed()) return List.of(new Candidate("NONE", 0));
        LinkedHashSet<Candidate> set = new LinkedHashSet<>();
        if (priority.priority() != DataPriority.CRITICAL || dataLength <=
CRITICAL_NO_COMPRESSION_MAX_BYTES) {
            set.add(new Candidate("NONE", 0));
        }
        switch (priority.priority()) {
            case CRITICAL -> {
                set.add(new Candidate("LZ4", 0));
                set.add(new Candidate("SNAPPY", 0));
                set.add(new Candidate("ZSTD", 1));
                set.add(new Candidate("ZSTD", 3));
            }
            case IMPORTANT -> {
                set.add(new Candidate("LZ4", 0));
                set.add(new Candidate("ZSTD", 1));
                set.add(new Candidate("ZSTD", 3));
                set.add(new Candidate("GZIP", 1));
            }
            case NORMAL -> {
                set.add(new Candidate("LZ4", 0));
                set.add(new Candidate("ZSTD", 1));
                set.add(new Candidate("ZSTD", 3));
                set.add(new Candidate("ZSTD", 6));
                set.add(new Candidate("GZIP", 1));
                set.add(new Candidate("GZIP", 6));
                if (recognition.modality() == DataModality.TEXT_LOG
                        || recognition.modality() ==
DataModality.STATISTICAL_REPORT) {
                    set.add(new Candidate("BROTLI", 6));
                }
            }
        }
        return List.copyOf(set);
    }
    private byte[] representativeSample(byte[] data) {
        int size = Math.min(sampleBytes, data.length);
        if (size == data.length) return data.clone();
        byte[] out = new byte[size];
        int third = size / 3;
        System.arraycopy(data, 0, out, 0, third);
        int middle = Math.max(0, data.length / 2 - third / 2);
        System.arraycopy(data, middle, out, third, third);
        int rest = size - third * 2;
        System.arraycopy(data, data.length - rest, out, third * 2, rest);
        return out;
    }
    private record Candidate(String algorithm, int level) {}
}
package com.datacompress.control.service;
import com.datacompress.control.domain.*;
import org.springframework.stereotype.Service;
```

```
import java.time.Instant;
import java.util.Locale;
@Service
public class DataClassificationService {
    public PriorityDecision decide(RecognitionResult recognition, String scene,
                                   String override, long producedAtMillis) {
        DataPriority priority;
        if (override != null && !override.isBlank() && !"AUTO".equalsIgnoreCase(override)) {
            priority = DataPriority.valueOf(override.toUpperCase(Locale.ROOT));
        } else if (recognition.modality() == DataModality.ALARM_EVENT
                || recognition.modality() == DataModality.CONTROL_MESSAGE)
{
            priority = DataPriority.CRITICAL;
        } else if (recognition.modality() == DataModality.DEVICE_OPERATION
                || recognition.modality() == DataModality.NUMERIC_TIME_SERIES)
{
            priority = DataPriority.IMPORTANT;
        } else {
            priority = DataPriority.NORMAL;
        }
        long now = Instant.now().toEpochMilli();
        long maxWait = switch (priority) {
            case CRITICAL -> 1000L;
            case IMPORTANT -> 10000L;
            case NORMAL -> 120000L;
        };
        int value = switch (priority) {
            case CRITICAL -> 100;
            case IMPORTANT -> 60;
            case NORMAL -> 20;
        };
        boolean cache = priority != DataPriority.CRITICAL;
        boolean batch = priority == DataPriority.NORMAL;
        String compression = switch (priority) {
            case CRITICAL -> "不压缩或 LZ4/SNAPPY/ZSTD-1 快速压缩";
            case IMPORTANT -> "LZ4/ZSTD/GZIP 平衡策略";
            case NORMAL -> "允许缓存、批量与较高压缩级别";
        };
        String failure = priority == DataPriority.CRITICAL
                ? "立即重传并触发严重告警"
                : priority == DataPriority.IMPORTANT
                ? "优先补传，超过截止时间告警"
                : "缓存后按优先级延迟补传";
        return new PriorityDecision(priority, value, now + maxWait, maxWait,
                cache, batch, compression, failure,
                "依据数据模态、业务场景和管理员覆盖规则分级");
    }
}
package com.datacompress.control.service;
import com.datacompress.control.domain.DataSourceInfo;
import org.springframework.stereotype.Service;
import java.time.Instant;
```

```
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
@Service
public class DataSourceService {
    private final Map<String, DataSourceInfo> sources = new ConcurrentHashMap<>();
    public DataSourceService() {
        save(new DataSourceInfo("sensor-line-01", "产线传感器", "SENSOR",
                "mqtt://edge-gateway/line1", "GW-L1", "设备监测",
                true, Instant.now(), 0L, 100));
        save(new DataSourceInfo("camera-yard-01", "厂区摄像头", "CAMERA",
                "rtsp://camera/yard1", "CAM-Y1", "视频巡检",
                true, Instant.now(), 0L, 1000));
        save(new DataSourceInfo("log-center", "日志系统", "LOG_SYSTEM",
                "http://log-center/api", "LOG-01", "运行审计",
                true, Instant.now(), 0L, 1000));
    }
    public List<DataSourceInfo> list() {
        return sources.values().stream()
                .sorted(Comparator.comparing(DataSourceInfo::name))
                .toList();
    }
    public DataSourceInfo get(String id) {
        return Optional.ofNullable(sources.get(id))
                .orElseThrow(() -> new IllegalArgumentException("数据源不存在: " + id));
    }
    public DataSourceInfo save(DataSourceInfo input) {
        String id = input.id() == null || input.id().isBlank()
                ? UUID.randomUUID().toString()
                : input.id();
        DataSourceInfo value = new DataSourceInfo(
                id, input.name(), input.type(), input.endpoint(),
                input.deviceCode(), input.businessScene(), input.online(),
                input.lastReportTime() == null ? Instant.now() : input.lastReportTime(),
                input.cumulativeBytes(), input.collectionPeriodMs());
        sources.put(id, value);
        return value;
    }
    public void addBytes(String id, long bytes) {
        sources.computeIfPresent(id, (key, old) -> new DataSourceInfo(
                old.id(), old.name(), old.type(), old.endpoint(), old.deviceCode(),
                old.businessScene(), true, Instant.now(),
                old.cumulativeBytes() + Math.max(0L, bytes), old.collectionPeriodMs()));
    }
}
package com.datacompress.control.service;
import com.datacompress.control.domain.*;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import java.util.Comparator;
import java.util.List;
```

```
@Service
public class DynamicStrategyService {
    private final double switchGainPercent;
    public DynamicStrategyService(
            @Value("${app.strategy.switch-gain-percent:3.0}") double switchGainPercent) {
        this.switchGainPercent = switchGainPercent;
    }
    public CompressionDecision select(
            List<AlgorithmProfile> profiles,
            RecognitionResult recognition,
            PriorityDecision priority,
            NetworkSample network,
            RateForecast forecast,
            long chunkBytes,
            CompressionDecision previous,
            long heldMillis) {
        if (network.quality() == NetworkQuality.DISCONNECTED) {
            boolean cache = priority.cacheAllowed();
            return new CompressionDecision("NONE", "NONE", 0,
                    chunkSize(network, priority), cache, priority.batchAllowed(),
                    sendOrder(priority), 0, 0, 0, Double.POSITIVE_INFINITY, 1.0,
                    cache ? "链路断开，任务写入缓存并保留断点" : "关键数据等待链路恢复后立即发送");
        }
        double rate = Math.max(1.0, forecast.safeRateKbps());
        CompressionDecision best = profiles.stream()
                .filter(AlgorithmProfile::available)
                .map(p -> evaluate(p, recognition, priority, network, rate, chunkBytes))
                .min(Comparator.comparingDouble(CompressionDecision::estimatedTotalMil
lis))
                .orElseThrow(() -> new IllegalStateException("没有可用压缩策略"));
        if (previous == null || previous.cache()) return best;
        if (previous.strategyKey().equals(best.strategyKey())) return best;
        AlgorithmProfile previousProfile = profiles.stream()
                .filter(p -> p.available() && p.algorithm().equals(previous.algorithm())
                        && p.level() == previous.level())
                .findFirst().orElse(null);
        if (previousProfile == null) return best;
        CompressionDecision current = evaluate(previousProfile, recognition, priority,
                network, rate, chunkBytes);
        double gain = (current.estimatedTotalMillis() - best.estimatedTotalMillis())
                / Math.max(0.001, current.estimatedTotalMillis()) * 100.0;
        if (heldMillis < 1000) {
            return copyWithReason(current, "尚未达到最小策略保持时间，避免频繁切换");
        }
        if (gain < switchGainPercent) {
            return copyWithReason(current, "候选策略预计收益 "
                    + String.format("%.2f", gain) + "%，未达到切换阈值");
        }
        return copyWithReason(best, best.reason() + "；相对当前策略预计节省 "
                + String.format("%.2f", gain) + "%");
    }
```

```
    private CompressionDecision evaluate(AlgorithmProfile profile,
                                         RecognitionResult recognition,
                                         PriorityDecision priority,
                                         NetworkSample network,
                                         double safeRateKbps,
                                         long chunkBytes) {
        long compressed = profile.compressedBytes(chunkBytes);
        double compression = profile.compressionMillis(chunkBytes);
        double transfer = compressed / 1024.0 / safeRateKbps * 1000.0;
        double decompression = profile.decompressionMillis(chunkBytes);
        double retryCost = transfer * network.packetLossPercent() / 100.0;
        double queuePenalty = priority.priority() == DataPriority.CRITICAL ? 0
                : priority.priority() == DataPriority.IMPORTANT ? 4 : 12;
        double total = compression + transfer + decompression + retryCost + queuePenalty;
        String preprocess = recognition.alreadyCompressed() ? "NONE" : recognition.suggestedPreprocess();
        return new CompressionDecision(preprocess, profile.algorithm(), profile.level(),
                chunkSize(network, priority), false, priority.batchAllowed(),
                sendOrder(priority), compression, transfer, decompression, total,
                profile.compressedRatio(),
                "最小化预处理、压缩、预测传输、解压、排队与重传代价");
    }
    private int chunkSize(NetworkSample network, PriorityDecision priority) {
        if (priority.priority() == DataPriority.CRITICAL) {
            return switch (network.quality()) {
                case GOOD -> 512 * 1024;
                case FLUCTUATING, RECOVERING -> 256 * 1024;
                case CONGESTED, DISCONNECTED -> 128 * 1024;
            };
        }
        return switch (network.quality()) {
            case GOOD -> 512 * 1024;
            case FLUCTUATING, RECOVERING -> 256 * 1024;
            case CONGESTED, DISCONNECTED -> 64 * 1024;
        };
    }
    private int sendOrder(PriorityDecision p) {
        return switch (p.priority()) {
            case CRITICAL -> 1;
            case IMPORTANT -> 2;
            case NORMAL -> 3;
        };
    }
    private CompressionDecision copyWithReason(CompressionDecision d, String reason) {
        return new CompressionDecision(d.preprocess(), d.algorithm(), d.level(),
                d.chunkSizeBytes(), d.cache(), d.batch(), d.sendOrder(),
                d.estimatedCompressionMillis(), d.estimatedTransferMillis(),
                d.estimatedDecompressionMillis(), d.estimatedTotalMillis(),
                d.estimatedCompressedRatio(), reason);
    }
}
```

```
package com.datacompress.control.service;
import com.datacompress.control.domain.*;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.*;
import java.util.concurrent.ConcurrentLinkedDeque;
import java.util.concurrent.atomic.AtomicLong;
@Service
public class NetworkTelemetryService {
    private final Deque<NetworkSample> history = new
ConcurrentLinkedDeque<>();
    private final Random random = new Random(20260625L);
    private final int historyLimit;
    private final long staleAfterMillis;
    private final double minimumFallbackRateKbps;
    private final AtomicLong tick = new AtomicLong();
    private volatile boolean simulationEnabled;
    private volatile NetworkSample externalSample;
    private volatile long externalReceivedAt;
    private volatile long lastMeasurementAt;
    private volatile String activeSource;
    private volatile double jitterEma;
    private volatile double previousRtt;
    public NetworkTelemetryService(
            @Value("${app.network.history-limit:7200}") int historyLimit,
            @Value("${app.network.simulation-enabled:true}") boolean simulationEnabled,
            @Value("${app.network.measurement-stale-after-ms:10000}") long staleAfterMillis,
            @Value("${app.network.minimum-fallback-rate-kbps:32}") double
minimumFallbackRateKbps) {
        this.historyLimit = Math.max(120, historyLimit);
        this.simulationEnabled = simulationEnabled;
        this.staleAfterMillis = Math.max(1000, staleAfterMillis);
        this.minimumFallbackRateKbps = Math.max(1.0,
minimumFallbackRateKbps);
        if (simulationEnabled) {
            for (int i = 0; i < 60; i++) append(simulated(i));
            markMeasured("SIMULATION");
        } else {
            append(idleSample());
            markMeasured("INTERNAL_IDLE");
        }
    }
    @Scheduled(fixedDelayString = "${app.network.sample-period-ms:500}")
    public void sample() {
        NetworkSample value;
        if (externalSample != null && System.currentTimeMillis() - externalReceivedAt < 3000) {
            value = externalSample;
            lastMeasurementAt = externalReceivedAt;
            activeSource = "EXTERNAL_TELEMETRY";
        } else if (simulationEnabled) {
            value = simulated(tick.incrementAndGet());
```

```
                markMeasured("SIMULATION");
            } else {
                value = idleSample();
                markMeasured("INTERNAL_IDLE");
            }
            append(value);
        }
        public NetworkSample current() {
            NetworkSample sample = history.peekLast();
            return sample == null ? idleSample() : sample;
        }
        public List<NetworkSample> history(int limit) {
            List<NetworkSample> values = new ArrayList<>(history);
            int from = Math.max(0, values.size() - Math.max(1, Math.min(limit, historyLimit)));
            return List.copyOf(values.subList(from, values.size()));
        }
        public List<Double> rateHistory(int limit) {
            return history(limit).stream().map(NetworkSample::throughputKbps)
                    .filter(v -> Double.isFinite(v) && v > 0).toList();
        }
        public void ingest(NetworkSample sample) {
            externalSample = new NetworkSample(
                    sample.timestamp() == null ? Instant.now() : sample.timestamp(),
                    positive(sample.throughputKbps(), 1), positive(sample.rttMillis(), 1),
                    Math.max(0, sample.jitterMillis()), clamp(sample.packetLossPercent(), 0, 100),
                    Math.max(0, sample.retransmissions()),
                    sample.connectionState() == null ? ConnectionState.CONNECTED :
sample.connectionState(),
                    sample.quality() == null ? classify(sample.throughputKbps(), sample.rttMillis(),
                            sample.jitterMillis(), sample.packetLossPercent(), sample.connectionState()) :
sample.quality(),
                    Math.max(0, sample.queuedBytes()), Math.max(0,
sample.cachedTasks()),
                    clamp(sample.cpuPercent(), 0, 100), clamp(sample.memoryPercent(), 0, 100), true);
            externalReceivedAt = System.currentTimeMillis();
            lastMeasurementAt = externalReceivedAt;
            activeSource = "EXTERNAL_TELEMETRY";
            simulationEnabled = false;
            append(externalSample);
        }
        /** Compatibility entry point for the V7 prediction module's ACK-confirmed measurements. */
        public synchronized NetworkSample recordTransfer(double
throughputKbps,
                                                         double rttMillis,
                                                         int retries,
                                                         long queuedBytes,
                                                         int cachedTasks) {
            double rate = positive(throughputKbps, minimumFallbackRateKbps);
            double rtt = positive(rttMillis, 1.0);
            double delta = previousRtt <= 0 ? 0 : Math.abs(rtt - previousRtt);
            jitterEma = jitterEma == 0 ? delta : 0.25 * delta + 0.75 * jitterEma;
            previousRtt = rtt;
            double loss = retries <= 0 ? 0 : Math.min(100.0, retries * 100.0 / (retries + 1.0));
            ConnectionState state = retries > 2 ? ConnectionState.CONGESTED
```

```
                : retries > 0 ? ConnectionState.UNSTABLE :
ConnectionState.CONNECTED;
        NetworkSample sample = new NetworkSample(Instant.now(), rate, rtt, jitterEma, loss,
                Math.max(0, retries), state, classify(rate, rtt, jitterEma, loss, state),
                Math.max(0, queuedBytes), Math.max(0, cachedTasks), 0, 0, true);
        markMeasured("INTERNAL_TCP_ACK");
        append(sample);
        return sample;
    }
    public synchronized NetworkSample recordFailure(long queuedBytes, int cachedTasks) {
        NetworkSample previous = current();
        NetworkSample sample = new NetworkSample(Instant.now(),
                Math.max(minimumFallbackRateKbps, previous.throughputKbps() *
0.55),
                Math.max(previous.rttMillis(), 1000),
                Math.max(previous.jitterMillis(), 100),
                Math.max(previous.packetLossPercent(), 50),
                previous.retransmissions() + 1,
                ConnectionState.CONGESTED, NetworkQuality.CONGESTED,
                Math.max(0, queuedBytes), Math.max(0, cachedTasks), 0, 0, true);
        markMeasured("INTERNAL_TCP_FAILURE");
        append(sample);
        return sample;
    }
    public TelemetryStatus status() {
        long age = telemetryAgeMillis();
        boolean fresh = lastMeasurementAt > 0 && age <= staleAfterMillis;
        List<Double> minute = rateHistorySince(60_000L);
        List<Double> fiveMinutes = rateHistorySince(5 * 60_000L);
        List<Double> oneHour = rateHistorySince(60 * 60_000L);
        double fallback = Math.max(minimumFallbackRateKbps,
current().throughputKbps());
        double p20 = percentile(minute, 0.20, fallback);
        double averageMinute = average(minute, fallback);
        double averageFive = average(fiveMinutes, averageMinute);
        double averageHour = average(oneHour, averageFive);
        double conservative = Math.max(minimumFallbackRateKbps, Math.min(p20, averageMinute *
0.85));
        return new TelemetryStatus(Instant.now(), fresh, age,
                activeSource == null ? "UNKNOWN" : activeSource,
                conservative, p20, averageMinute, averageFive, averageHour, history.size());
    }
    public boolean isTelemetryFresh() {
        return lastMeasurementAt > 0 && telemetryAgeMillis() <= staleAfterMillis;
    }
    public long telemetryAgeMillis() {
        if (lastMeasurementAt <= 0) return Long.MAX_VALUE;
        return Math.max(0, System.currentTimeMillis() - lastMeasurementAt);
    }
    public void setSimulationEnabled(boolean enabled) {
        simulationEnabled = enabled;
        if (enabled) {
            externalSample = null;
            externalReceivedAt = 0;
```

```
                markMeasured("SIMULATION");
            }
        }
        public boolean isSimulationEnabled() { return simulationEnabled; }
        private void markMeasured(String source) {
            lastMeasurementAt = System.currentTimeMillis();
            activeSource = source;
        }
        private NetworkSample idleSample() {
            return new NetworkSample(Instant.now(), 1200, 35, 5, 0.1,
                    0, ConnectionState.CONNECTED, NetworkQuality.GOOD,
                    0L, 0, 30, 45, false);
        }
        private NetworkSample simulated(long n) {
            double phase = n % 180;
            double base = 1500 + 350 * Math.sin(n / 18.0) + 160 * Math.sin(n / 5.5);
            if (phase > 65 && phase < 95) base -= 850;
            if (phase > 125 && phase < 138) base -= 1200;
            double rate = Math.max(50, base + random.nextGaussian() * 45);
            double loss = phase > 125 && phase < 138 ? 8 + random.nextDouble() * 8
                    : phase > 65 && phase < 95 ? 2 + random.nextDouble() * 4
                    : Math.max(0, random.nextGaussian() * 0.4);
            double rtt = 22 + Math.max(0, 1600 - rate) / 7 + random.nextDouble() * 10;
            double jitter = 3 + loss * 2.2 + random.nextDouble() * 8;
            ConnectionState state = rate < 80 ? ConnectionState.DISCONNECTED
                    : rate < 450 ? ConnectionState.CONGESTED
                    : loss > 3 ? ConnectionState.UNSTABLE
                    : ConnectionState.CONNECTED;
            NetworkQuality quality = classify(rate, rtt, jitter, loss, state);
            return new NetworkSample(Instant.now(), rate, rtt, jitter, loss,
                    loss > 1 ? (int)Math.ceil(loss / 2) : 0, state, quality,
                    0L, 0, 28 + random.nextDouble() * 22,
                    35 + random.nextDouble() * 20, false);
        }
        private NetworkQuality classify(double rate, double rtt, double jitter,
                                        double loss, ConnectionState state) {
            if (state == ConnectionState.DISCONNECTED || rate < 80) return
NetworkQuality.DISCONNECTED;
            if (state == ConnectionState.RECOVERING) return
NetworkQuality.RECOVERING;
            if (rate < 500 || loss > 5 || rtt > 250) return NetworkQuality.CONGESTED;
            if (loss > 1.5 || jitter > 35 || rtt > 120) return NetworkQuality.FLUCTUATING;
            return NetworkQuality.GOOD;
        }
        private List<Double> rateHistorySince(long millis) {
            Instant cutoff = Instant.now().minusMillis(Math.max(1, millis));
            return history.stream()
                    .filter(sample -> sample.timestamp() != null && !sample.timestamp().isBefore(cutoff))
                    .map(NetworkSample::throughputKbps)
                    .filter(value -> Double.isFinite(value) && value > 0)
                    .toList();
        }
```

```
    private void append(NetworkSample sample) {
        history.addLast(sample);
        while (history.size() > historyLimit) history.pollFirst();
    }
    private double average(List<Double> values, double fallback) {
        return values.stream().mapToDouble(Double::doubleValue).average().orElse(fallback);
    }
    private double percentile(List<Double> values, double quantile, double fallback) {
        if (values.isEmpty()) return fallback;
        List<Double> sorted = new ArrayList<>(values);
        sorted.sort(Double::compareTo);
        int index = (int)Math.floor(Math.max(0, Math.min(1, quantile)) * (sorted.size() - 1));
        return sorted.get(index);
    }
    private double positive(double value, double fallback) {
        return Double.isFinite(value) && value > 0 ? value : fallback;
    }
    private double clamp(double value, double min, double max) {
        return Math.max(min, Math.min(max, value));
    }
}
package com.datacompress.control.service;
import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.util.*;
import java.util.zip.ZipEntry;
import java.util.zip.ZipInputStream;
import java.util.zip.ZipOutputStream;
@Service
public class PackagePayloadService {
    public static final String MAGIC = "DCCS-PACKAGE-V1";
    public PackagedPayload build(List<MultipartFile> uploads) throws IOException {
        return build(uploads, null, null);
    }
    public PackagedPayload build(List<MultipartFile> uploads, String requestedName, String requestedKind) throws IOException {
        if (uploads == null || uploads.isEmpty()) {
            throw new IllegalArgumentException("请至少选择一个待压缩文件或目录包");
        }
        List<PackageEntrySource> entries = new ArrayList<>();
        Set<String> usedNames = new HashSet<>();
        long rawBytes = 0;
        int index = 1;
        for (MultipartFile upload : uploads) {
            if (upload == null || upload.isEmpty()) continue;
            String entryPath = normalizePath(upload.getOriginalFilename());
            if (entryPath.isBlank()) entryPath = "file-" + index;
            entryPath = uniquePath(entryPath, usedNames);
            long size = upload.getSize();
```

```
                if (size <= 0) continue;
                if (size > Integer.MAX_VALUE) {
                    throw new IllegalArgumentException("单个文件超过当前任务包格式限制");
                }
                rawBytes += size;
                entries.add(new PackageEntrySource(entryPath, upload, size));
                index++;
            }
            if (entries.isEmpty() || rawBytes == 0) {
                throw new IllegalArgumentException("文件不能为空");
            }
            if (rawBytes > Integer.MAX_VALUE - 1024L * 1024L) {
                throw new IllegalArgumentException("任务包超过当前内存打包限制");
            }
            ByteArrayOutputStream buffer = new ByteArrayOutputStream(initialPackageCapacity(rawBytes,
entries));
            try (DataOutputStream out = new DataOutputStream(buffer)) {
                out.writeUTF(MAGIC);
                out.writeInt(entries.size());
                for (PackageEntrySource entry : entries) {
                    out.writeUTF(entry.path());
                    out.writeInt(Math.toIntExact(entry.size()));
                    try (InputStream input = entry.upload().getInputStream()) {
                        long copied = input.transferTo(out);
                        if (copied != entry.size()) {
                            throw new EOFException("文件读取不完整: " + entry.path());
                        }
                    }
                }
            }
            List<String> entryPaths = entries.stream().map(PackageEntrySource::path).toList();
            String packageKind = determineKind(entryPaths, requestedKind);
            String packageName = determineName(entryPaths, requestedName,
packageKind);
            return new PackagedPayload(packageName, packageKind,
"application/vnd.dccs.package",
                    buffer.toByteArray(), rawBytes, entries.size());
        }
        public boolean isPackage(byte[] payload) {
            if (payload == null || payload.length < 4) return false;
            try (DataInputStream in = new DataInputStream(new ByteArrayInputStream(payload))) {
                return MAGIC.equals(in.readUTF());
            } catch (IOException ignored) {
                return false;
            }
        }
        public List<PackageEntryData> readAnyEntries(byte[] payload) throws IOException {
            if (isPackage(payload)) return readEntries(payload);
            return readZipEntries(payload);
        }
        public byte[] toViewableZip(byte[] packagePayload) throws IOException {
            List<PackageEntryData> entries = readEntries(packagePayload);
            ByteArrayOutputStream buffer = new ByteArrayOutputStream();
```

```
        try (ZipOutputStream zip = new ZipOutputStream(buffer)) {
            for (PackageEntryData entry : entries) {
                ZipEntry zipEntry = new ZipEntry(entry.path());
                zip.putNextEntry(zipEntry);
                zip.write(entry.bytes());
                zip.closeEntry();
            }
        }
        return buffer.toByteArray();
    }
    public byte[] buildCompressedZipView(TransferTaskEntity task, byte[] dcczPayload) throws IOException {
        ByteArrayOutputStream buffer = new ByteArrayOutputStream();
        try (ZipOutputStream zip = new ZipOutputStream(buffer)) {
            putText(zip, "README.txt", String.join("\n",
                    "这是系统生成的可打开压缩包，用于测试后端真实压缩流程。",
                    "compressed-payload.dccz 是后端分块压缩后的二进制载荷，不应当用记事本直接打
开。",
                    "metadata.json ，，， SHA-256 校验值。",
                    "恢复后的业务文件可从系统任务队列中的“打开文件”或“打开文件夹”查看。",
                    ""));
            putText(zip, "metadata.json", metadataJson(task));
            ZipEntry payload = new ZipEntry("compressed-payload.dccz");
            zip.putNextEntry(payload);
            zip.write(dcczPayload);
            zip.closeEntry();
        }
        return buffer.toByteArray();
    }
    private String metadataJson(TransferTaskEntity task) {
        String algorithm = task.decision == null ? "-" : task.decision.algorithm() + "-" + task.decision.level();
        return "{\n"
                + "  \"packageName\": \"" + json(task.fileName) + "\",\n"
                + "  \"packageKind\": \"" + json(task.packageKind) + "\",\n"
                + "  \"itemCount\": " + task.itemCount + ",\n"
                + "  \"originalPackageBytes\": " + task.originalBytes + ",\n"
                + "  \"compressedPayloadBytes\": " + task.compressedBytes + ",\n"
                + "  \"algorithm\": \"" + json(algorithm) + "\",\n"
                + "  \"chunks\": " + task.completedChunks + ",\n"
                + "  \"checksum\": \"" + json(task.checksum) + "\",\n"
                + "  \"integrityPassed\": " + task.integrityPassed + "\n"
                + "}\n";
    }
    private void putText(ZipOutputStream zip, String name, String text) throws IOException {
        zip.putNextEntry(new ZipEntry(name));
        zip.write(text.getBytes(StandardCharsets.UTF_8));
        zip.closeEntry();
    }
    public List<PackageEntryData> readEntries(byte[] payload) throws IOException {
        try (DataInputStream in = new DataInputStream(new ByteArrayInputStream(payload))) {
            String magic = in.readUTF();
            if (!MAGIC.equals(magic)) throw new IOException("不是有效的 DCCS 包载荷");
```

```
            int count = in.readInt();
            if (count < 0 || count > 100_000) throw new IOException("包条目数量非法");
            List<PackageEntryData> entries = new ArrayList<>();
            for (int i = 0; i < count; i++) {
                String entryPath = normalizePath(in.readUTF());
                int length = in.readInt();
                if (length < 0) throw new IOException("包条目长度非法");
                byte[] bytes = in.readNBytes(length);
                if (bytes.length != length) throw new EOFException("包条目读取不完整");
                entries.add(new PackageEntryData(entryPath, bytes));
            }
            return List.copyOf(entries);
        }
    }
    private List<PackageEntryData> readZipEntries(byte[] payload) throws IOException {
        List<PackageEntryData> entries = new ArrayList<>();
        try (ZipInputStream zip = new ZipInputStream(new ByteArrayInputStream(payload))) {
            ZipEntry entry;
            while ((entry = zip.getNextEntry()) != null) {
                if (entry.isDirectory()) continue;
                String entryPath = normalizePath(entry.getName());
                if (entryPath.isBlank()) continue;
                ByteArrayOutputStream buffer = new ByteArrayOutputStream();
                zip.transferTo(buffer);
                entries.add(new PackageEntryData(entryPath, buffer.toByteArray()));
                zip.closeEntry();
            }
        }
        if (entries.isEmpty()) throw new IOException("任务内容为空或不是可查看的文件包");
        return List.copyOf(entries);
    }
    private String determineKind(List<String> entryPaths, String requestedKind) {
        if ("FOLDER".equalsIgnoreCase(requestedKind)) return "FOLDER";
        if ("FILE".equalsIgnoreCase(requestedKind) && entryPaths.size() == 1) return "FILE";
        return entryPaths.size() > 1 || entryPaths.stream().anyMatch(path -> path.contains("/")) ? "FOLDER" : "FILE";
    }
    private String determineName(List<String> entryPaths, String requestedName, String packageKind) {
        String requested = sanitizeDisplayName(requestedName);
        if (!requested.isBlank()) return requested;
        String firstPath = entryPaths.get(0);
        if ("FOLDER".equals(packageKind) && firstPath.contains("/")) {
            return sanitizeDisplayName(firstPath.substring(0, firstPath.indexOf('/')));
        }
        int slash = firstPath.lastIndexOf('/');
        return sanitizeDisplayName(slash >= 0 ? firstPath.substring(slash + 1) : firstPath);
    }
    private int initialPackageCapacity(long rawBytes, List<PackageEntrySource> entries) {
        long overhead = 16L + entries.stream().mapToLong(entry -> entry.path().length() * 3L + 8L).sum();
        long capacity = Math.min(Integer.MAX_VALUE - 8L, Math.max(1024L, rawBytes +
overhead));
        return Math.toIntExact(capacity);
```

```
    }
    private String normalizePath(String original) {
        if (original == null) return "";
        String value = original.replace('\\', '/').trim();
        value = value.replaceAll("^[A-Za-z]:", "");
        while (value.startsWith("/")) value = value.substring(1);
        List<String> parts = new ArrayList<>();
        for (String raw : value.split("/")) {
            String part = raw.trim().replaceAll("[\\\\:*?\"<>|]", "_");
            if (part.isBlank() || part.equals(".") || part.equals("..")) continue;
            parts.add(part);
        }
        return String.join("/", parts);
    }
    private String uniquePath(String path, Set<String> used) {
        String value = path;
        int counter = 2;
        while (!used.add(value)) {
            int dot = path.lastIndexOf('.');
            if (dot > 0) {
                value = path.substring(0, dot) + "-" + counter + path.substring(dot);
            } else {
                value = path + "-" + counter;
            }
            counter++;
        }
        return value;
    }
    private String sanitizeDisplayName(String name) {
        String value = name == null ? "" : name.trim();
        value = value.replaceAll("[\\\\/:*?\"<>|]", "_");
        return value;
    }
    private String json(String value) {
        return String.valueOf(value == null ? "" : value)
                .replace("\\", "\\\\")
                .replace("\"", "\\\"")
                .replace("\n", "\\n")
                .replace("\r", "\\r");
    }
    private record PackageEntrySource(String path, MultipartFile upload, long size) {}
    public record PackageEntryData(String path, byte[] bytes) {
        public PackageEntryData {
            path = path == null ? "" : path;
            bytes = bytes == null ? new byte[0] : bytes;
        }
    }
    public record PackagedPayload(String fileName, String packageKind, String mimeType, byte[] payload,
                                  long rawFileBytes, int entryCount) {}
}
```